

控制权、进程、线程与调度

沿进程生命周期理解 408 核心机制

408 操作系统专题手册

核心问题

用户程序怎样高速直接运行，而操作系统又始终能够重新取得控制权，并把有限 CPU 时间安全地分给多个执行流？

本册沿用整个 OS 知识体系的统一推演引擎：

State → Transition → Failure (Bad State) → Property (Safety/Liveness)
→ Minimal Mechanism → Tradeoff

对本专题而言，最常见的状态转移可以继续写成：

当前状态 + 事件 $\xrightarrow[\text{策略}]{\text{机制}}$ 新状态

真正要检查的不是“背出了哪个状态图”，而是：这次事件改变了什么状态？什么坏状态必须被排除？内核用什么最小机制重新取得控制并维持安全性与进展性？

本册 Owner / Stop Boundary

本册只完整拥有：task、执行上下文、进程/线程状态、run/wait queue、CPU 调度、block/wakeup 与控制权交接。

同步协议、PV/管程与死锁由 OS-03 拥有；页表与换页由 OS-04 拥有；OFD、dentry/inode 与文件生命周期由 OS-06/07 拥有；设备请求与 DMA 完成路径由 OS-05 拥有。本册在 fork/read/exec 中只保留与进程状态或资源引用交接直接相关的接口，不在这里重讲后续 Topic 的内部机制。

目录

1 核心模型：控制权、执行实体与资源状态	2
1.1 五个检查维度与 9 个核心要素	2
1.2 两个不能混淆的“状态”	3
2 第零章：为什么内核必须先解决“控制权”问题	4
2.1 受限直接执行：又要快，又不能失控	4
2.2 用户态与内核态：不是“两个程序”，而是权限边界	4
2.3 进入内核的三种典型入口	5

2.4	Trap 之后不一定 Context Switch: 彻底攻克 408 核心四连辨析	5
3	程序、进程与线程: 把“静态描述、运行实例、资源世界和执行路径”拆开	7
3.1	Program、Process Image、Process Instance 与 Thread: 先把四层对象分开	7
3.2	母问题: 为什么需要将资源所有权与执行流解耦	7
3.3	独立推进必然推出 PC、寄存器与栈的独立私有	8
3.4	共享与私有: 三权分立模型 (Ownership $\neq$ Addressability $\neq$ Concurrency Safety)	8
3.5	线程局部存储 (TLS): 同一地址空间中的逻辑私有状态	9
3.6	共享文件描述符: 共享句柄意味着共享下游可变引用状态	9
3.7	并发、并行与 408 进程/线程全维度对照	10
4	PCB: 把“进程是什么”落实到内核可管理的数据结构	10
4.1	PCB 不是进程本体, 而是内核管理进程的控制记录	10
4.2	PCB 的组织方式: 本质是“怎样按身份/状态快速找到控制记录”	11
4.3	PCB 与线程上下文: 教材抽象和现代实现要分层	11
4.4	TCB: 把线程自己的执行状态单独拿出来	12
5	进程通信 IPC: 隔离之后怎样交换信息	12
5.1	母问题: 进程彼此隔离, 为什么还能通信?	12
5.2	四类典型 IPC: 看“共享什么、等什么”	13
5.3	共享内存为什么“快”, 又为什么不等于“简单”	14
6	进程资源接口: 本册只追踪“引用怎样随生命周期交接”	14
7	进程生命周期主线	15
7.1	生命周期总览	15
7.2	五态为什么还不够: 挂起与七态模型	15
7.3	状态不是一个标签: 它必须落实为 Queue Membership + Wait Relation	16
7.4	进程控制原语: 一次合法状态变化是原子状态事务	17
7.5	阶段 A: 创建——从父进程产生新执行环境	17
7.6	阶段 B: exec——不是新建进程, 而是替换进程映像	18
7.7	阶段 C: Ready——只缺 CPU	18
7.8	阶段 D: Running——用户态和内核态都可能属于“这个进程正在运行”	18
7.9	阶段 E: Blocked——即使给 CPU 也不能继续	18
7.10	阶段 F: Wakeup——Blocked $\rightarrow$ Ready, 不等于马上 Running	19
7.11	阶段 G: Exit / Zombie / Wait	19
7.12	线程生命周期、线程组 (G) 与事件作用域	19

8 调度：控制权已经回到内核，下一步给谁？	20
8.1 先分清三级调度：它们选择的不是同一种“下一位”	20
8.2 Scheduler $\neq$ Dispatcher：先决定，再真正交出 CPU	20
8.3 机制与策略必须拆开	21
8.4 调度评价指标是互相冲突的目标函数	21
8.5 先识别工作负载，再决定“什么叫好”	22
8.6 经典算法应按“偏好谁”来记	22
8.7 MLQ 与 MLFQ：把一条就绪队列拆开后，会新增哪三个决策？	22
8.8 HRRN：不要只背公式，要看“等待如何改变优先级”	23
8.9 公平性必须带主语：Process Fairness $\neq$ User Fairness $\neq$ Thread Fairness	23
8.10 时间片到底解决什么	24
8.11 多处理机调度：单 CPU 问“谁”，多 CPU 还必须问“放到哪里”	24
8.12 有调度原因，不等于“现在这一条指令就能切换”	25
9 上下文切换：从教材“三层上下文”映射到四维切换模型	25
9.1 教材说“上下文”时，不要自动缩成“寄存器”	25
9.2 四维切换模型：到底切了什么？	26
9.3 典型并发场景的四维判定矩阵	26
9.4 切换开销的深度解构：直接成本与间接成本	26
9.5 动态轨迹：A 切到 B	27
10 系统调用、中断、异常：控制流为什么会改变	27
10.1 中断响应要拆成两层：硬件自动入口与软件处理程序	27
10.2 一次系统调用的五阶段模板	28
10.3 时钟中断为什么是抢占式调度的基础	28
10.4 设备中断：唤醒并不等于直接交 CPU	28
11 综合题一：从 fork 到 wait 的进程生命周期	29
11.1 完整轨迹	29
12 综合题二：一次阻塞 read 的“进程侧投影”	29
13 线程实现模型：从“谁看得见”推出 ULT/KLT、M:N 与现代 Runtime	30
13.1 可见性原则：谁调度，谁就必须“看见”被调度的对象	30
13.2 ULT 与 KLT：不要问名字，问谁维护控制状态、谁执行选择	31
13.3 M:1、1:1、M:N 的本质：两层实体集合之间的映射	31
13.4 两级队列与“Ready 必须带主语”	32

13.5 为什么经典 M:N 在 OS 层面走向失败：信息分裂 (Information Split)	32
13.6 Scheduler Activations：跨层通知的经典桥梁	33
13.7 阻塞传播与实际并行度上界公式	33
13.8 现代 Runtime 的本质复活：执行上下文表示论 (<i>Object ≠ Representation</i>)	33
14 408 解题检查表：从概念辨析到状态推演	34
14.1 线程做题九问控制器 (Thread Control Adapter)	34
14.2 进程状态题八问	35
14.3 调度计算题九步	35
14.4 系统调用/中断题九问	36
14.5 PCB / TCB / 进程资源接口题八问	36
14.6 IPC 题六问	36
15 原笔记与教材中值得保留、但必须升格的内容	37
16 知识树与专题接口	38
参考与工程边界来源	39

1 核心模型：控制权、执行实体与资源状态

1.1 五个检查维度与 9 个核心要素

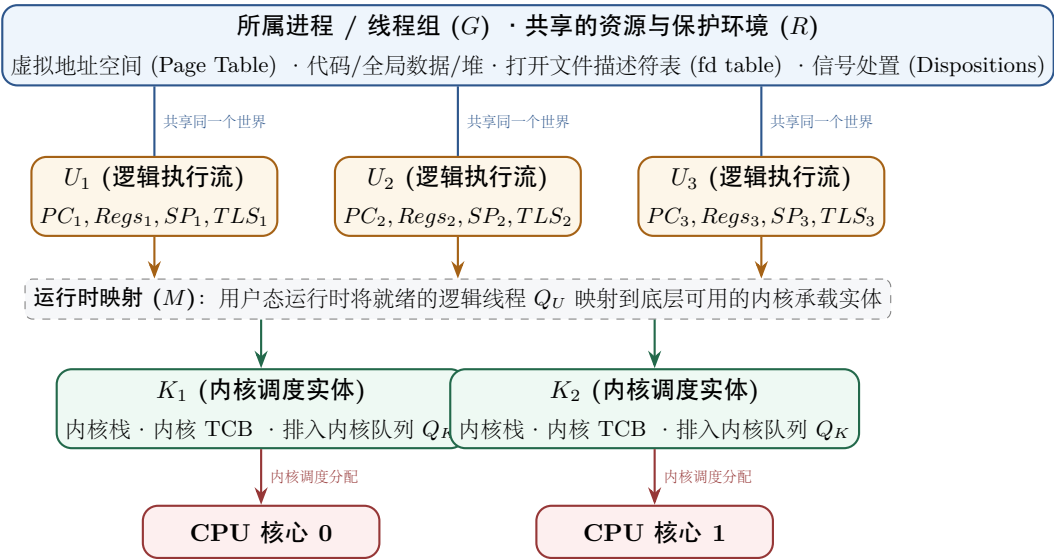
面对进程、线程、调度、系统调用和中断，先回答五个母问题：

- 1. 谁现在拥有 CPU 控制权？用户程序还是内核？
- 2. CPU 上真正正在执行的实体是谁？一个进程中的哪条执行流？
- 3. 这个执行流依附于哪个资源环境？地址空间、文件表、权限、信号等属于谁？
- 4. 什么事件会迫使执行路径改变？系统调用、异常、中断、阻塞、唤醒、抢占、退出。
- 5. 当多个执行实体都想运行时，谁来选择下一位？调度机制如何获得机会，调度策略如何作选择？

为了把所有零散的概念（PCB/TCB、ULT/KLT、M:N、阻塞传播、两级队列、TLS、文件共享）统一起来，我们不需要死记硬背名词，只需要在脑海中建立包含 **9 个核心要素**的完整关系图：

$$S_{\text{exec}} = (G, R, U, K, M, Q_U, Q_K, W, C)$$

要素	核心追问	涵盖的具体对象与状态
<i>R</i>	共享哪个资源与保护世界？	虚拟地址空间、代码段/全局数据/堆、打开文件描述符表、进程凭证
<i>U</i>	有哪些独立的逻辑执行流？	每条逻辑线程私有的现场： (<i>PC, Registers, SP, ExecutionState, TLS</i>)
<i>K</i>	内核能独立调度哪些实体？	Kernel Schedulable Contexts（内核 TCB、内核栈、内核调度属性）
<i>M</i>	逻辑线程怎样映射到内核实体？	映射关系 $M : \{U\} \rightarrow \{K\}$ （M:1 纯用户级、1:1 内核级、M:N 混合级）
<i>G</i>	这是哪个进程实例与生命周期/关系作用域？	PID/线程组身份、父子关系、退出与等待观察、进程级信号作用域，以及哪些 U_i 属于该进程实例
Q_U	用户运行时里谁处于就绪状态？	用户态就绪队列（User Runnable Queue）
Q_K	内核调度器里谁处于可运行状态？	内核可运行队列（Kernel Run Queue）
<i>W</i>	谁在等待什么事件或条件？	等待集合（等待磁盘 I/O、等待锁、等待线程退出 <code>join</code> 等）
<i>C</i>	物理 CPU 核心正在运行哪个实体？	CPU 载体与核心绑定情况



理解线程的四句核心硬道理

1. **线程 = 可独立恢复的执行流**: 只要要求能停在任意位置并独立继续 (Independent Progress), 就必须各自保存私有的 PC、寄存器与栈 (Resumable State);
2. **进程实例 = 身份/生命周期作用域 G + 资源/保护环境 R + 执行流集合 $\{U_i\}$** : 它不仅回答“共同生活在哪个资源世界”, 还回答“这是哪一次运行实例、与谁有生命周期关系、哪些执行流属于它”;
3. **逻辑线程 $\neq$ 内核调度实体**: 用户看到的线程 (U) 与内核能调度的实体 (K) 并不总是一回事。在 1:1 中两者合一, 在 M:1 和 M:N 中彻底分离;
4. **共享 $\neq$ 可以随意并发访问**: 同一个进程内的内存大家都能寻址到 (Addressability), 但不代表可以不加锁随意并发写 (Concurrency Safety)。

进程与调度八问

拿到任何进程/线程/调度题, 依次问:

1. 当前运行实体是谁?
2. 当前处于用户态还是内核态?
3. 发生了什么事件?
4. 这个事件只是**模式切换**, 还是会导致**调度/上下文切换**?
5. 当前实体为什么还能继续/为什么必须等待?
6. 若不能继续, 它应进入 Ready 还是 Blocked?
7. 哪些资源继续属于它, 哪些状态需要保存?
8. 最终谁获得 CPU, 代价是什么?

1.2 两个不能混淆的“状态”

本模块同时出现两类状态:

- **体系结构执行状态**：PC、通用寄存器、SP、状态寄存器等，决定“这条执行流下一步从哪里继续”。
- **内核管理状态**：任务状态、调度信息、地址空间引用、打开文件、权限、等待对象等，决定“OS 怎样管理它”。

进程/线程切换，本质上就是：保存旧执行上下文，选择新的可运行实体，再恢复新执行上下文。

2 第零章：为什么内核必须先解决“控制权”问题

2.1 受限直接执行：又要快，又不能失控

最简单的高性能方案当然是让用户程序直接在 CPU 上运行：

Program → CPU

但若程序执行：

```
while (1) { }
```

且 OS 没有任何硬件帮助，它可能永远拿不回 CPU。

因此产生核心矛盾：

Direct Execution for Performance

 vs.

Restricted Control for Safety

解决它依赖**特权级 + 陷入/返回机制 + 时钟中断**。

2.2 用户态与内核态：不是“两个程序”，而是权限边界

CPU 至少区分两类执行权限：

对比维度	用户态 (User Mode)	内核态 (Kernel Mode)
能否执行特权指令	受限（仅非特权指令）	可以（完整指令集）
能否直接配置页表/设备	通常不允许（引发硬件异常）	内核可以（拥有写 CR3 与 I/O 权限）
运行的代码	普通应用 / 用户函数库	内核代码 / 驱动程序
设计目标	隔离进程、限制非法破坏	虚拟化资源、管理全局安全

关键理解

用户态/内核态是 CPU 执行权限状态，不等于“当前是不是 OS 进程”。同一个用户进程发起系统调用后，可以在同一进程上下文中执行内核代码。

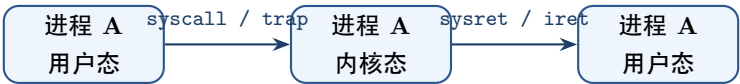
2.3 进入内核的三种典型入口

入口	同步性	典型来源	直观含义与响应动作
系统调用	同步	当前程序显式请求 (trap / syscall)	“我需要内核提供服务”
异常/陷阱	同步	当前指令执行产生 (除零 / 缺页)	“这条指令遇到了特殊硬件情况”
外部中断	异步	定时器、磁盘 I/O、网卡到达	“外部硬件设备发生了事件”

它们都可能使 CPU 从 user mode 进入 kernel mode，但原因不同、后续处理也不同。

2.4 Trap 之后不一定 Context Switch：彻底攻克 408 核心四连辨析

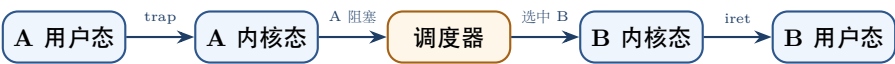
这是 408 最常考、也最容易混淆的经典概念区。很多初学者容易将“进入内核”、“发生中断”、“触发调度”与“进程切换”直接画等号。必须沿事件的执行路径，把这四组“不等于 (≠)”背后的底层机制、反例与触发条件彻底拆透：



这只是：

Mode Switch (模式切换：仅换特权级与内核栈，不换进程，不换页表)

若 A 在内核中因为等待 I/O 不能继续，才会触发调度并选择 B：



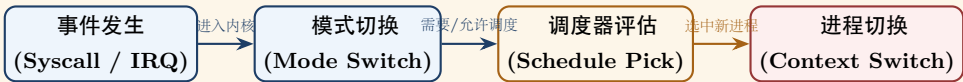
这才是：

Context Switch (进程上下文切换：换通用寄存器现场 + 换页表 CR3)

核心辨析点	为什么“不等于”？（机制与反例）	什么时候才会真正“发生”？
1. 进入内核态 ≠ 切换进程	模式切换 (Mode Switch) 只是 CPU 执行特权级的改变 (用户态 ↔ 内核态)，并换用进程私有的内核栈。进程的 PID、虚拟地址空间（页表基址 CR3）完全没变，CPU 依然在为原进程执行内核服务代码。 典型反例：getpid()、time() 或读已缓存数据的 read()，内核在几微秒内执行完毕，直接通过 sysret/iret 原路返回同一进程。	当前进程在内核中因 I/O、锁或信号量进入 Blocked （如读取磁盘块），或时间片耗尽被抢占，内核调度器才会把 CPU 换给另一个进程。
2. 发生中断 ≠ 进行调度	硬件中断 (如时钟滴答、网卡数据包到达) 只是给内核一次执行中断服务程序 (ISR) 的控制机会。ISR 负责处理硬件事件，处理完毕后会检查“是否需要调度”。 典型反例：RR 算法中时间片为 100ms，系统每 10ms 发生一次时钟中断。前 9 次中断仅仅是 ISR 累加进程运行时间 (runtime += 10ms)，因时间片未用尽且无更高优先级任务，ISR 直接 iret 原路返回当前进程，根本不进入调度器。	时钟中断检测到当前任务的时间片已经耗尽 ($q \leq 0$)； 外设中断唤醒了处于就绪队列中且优先级严格更高的任务（在抢占式系统中触发抢占）。
3. 进行调度 ≠ 换成别的进程	调度器 (Scheduler) 是一段评估与选择算法 (如遍历就绪队列)。执行 schedule() 并不意味着必须换人，当前进程本身也是就绪队列的候选者之一。 典型反例：系统中仅有进程 A 就绪 (其他任务都在等待 I/O)，或者优先级调度中进程 A 的优先级依然最高。调度器评估后再次选中 A，CPU 直接恢复 A 继续执行，跳过昂贵的跨进程切换与页表刷新。	调度算法根据策略选出了一个 不同于当前进程的新就绪实体 B （例如时间片轮转选中下一个，或当前进程已阻塞无法继续）。
4. 阻塞 (Blocked) → 必然引发调度	阻塞的本质是“即使给 CPU 也无法继续推进”（在等磁盘读完、等网络包到达或等锁释放）。操作系统决不能允许 CPU 在内核中白白空转死等，内核必须将当前进程置为 Blocked 并挂入等待队列，立刻主动调用调度器将 CPU 让给其他就绪任务。	只要发生真正的阻塞，系统就必然触发调度机会；若此时没有其他就绪用户进程，内核会调度执行专用的空闲进程 (idle task) 进入低功耗等待。

做题时的三步递进闸门模型

做 408 考题时，按以下三道闸门逐级推演，不要一步跳到结论：



任何一步条件不满足（如快速系统调用原路返回、时间片未耗尽、就绪队列依然选中自己），链条都会立刻提前终止并原路返回，绝不会盲目触发进程切换！

3 程序、进程与线程：把“静态描述、运行实例、资源世界和执行路径”拆开

3.1 Program、Process Image、Process Instance 与 Thread：先把四层对象分开

408 教材里“程序、进程、进程映像、线程”经常相邻出现，但它们并不处在同一层：

$$\text{Program / Executable Image} \xrightarrow{\text{create/load}} \text{Process Instance } P = (G, R, \{U_i\})$$

- **Program / Executable Image (程序/可执行映像)**：描述“要执行什么代码与静态内容”，本身没有 PID、Ready/Blocked、父子关系或正在等待的事件；
- **Process Instance (进程实例)**：OS 对“一次正在被管理的运行”建立的动态身份，包含生命周期/关系作用域 G 、资源/保护环境 R 与一条或多条执行流 $\{U_i\}$ ；
- **Process Image (进程映像/进程实体的教材表示)**：经典教材常用“PCB + 程序段 + 数据段（有的口径还单列内核栈等）”描述某一时刻进程在机器中的表示。它是 **Representation**，不是进程抽象本身；
- **Thread (线程)**：进程内部一条可独立保存、等待、恢复的执行路径。

最强的边界例子是 `exec`：

$$\text{same Process Identity } G \quad + \quad \text{new Program Image}$$

成功的 `exec` 可以替换当前运行的程序映像而不创建新的进程身份；反过来，同一个程序文件也可以产生多个不同 PID 的进程实例。因此：

$$\text{Program} \neq \text{Process Instance} \neq \text{Thread}, \quad \text{Process} \neq \text{PCB}$$

408 教材的“四个特征”怎样挂回母模型

教材常用**动态性**、**并发性**、**独立性**、**异步性**描述进程。不要把它们当四个孤立形容词：动态性来自“进程有创建—运行—等待—退出的生命周期”；多个动态实例可以并存并交错推进，于是体现并发性；OS 又必须为每个实例保存独立身份、资源关系与控制状态，于是体现独立性；多个实例受调度和外部事件驱动，推进速度不可预先固定，于是体现异步性。这里是一条**理解/记忆生成链**，不是数学上的严格逻辑蕴含。

3.2 母问题：为什么需要将资源所有权与执行流解耦

不要从机械背诵“线程是什么”开始。真正的问题是：

一个程序已经拥有虚拟地址空间、打开文件、权限等资源以后，怎样让多条执行流共享这个资源世界，却又能够各自执行、暂停、等待、恢复并最终获得有限 CPU 的执行机会？

单线程进程实际上将两件事紧密绑定在了一起：

$$\text{Resource Ownership (资源所有权)} + \text{Single Execution Flow (单一执行流)}$$

这导致了天然的工程局限：

- 若当前执行流因等待 I/O 或事件而阻塞，整个资源世界将无法推进其他独立计算；
- 即使多核 CPU 具备充沛的物理算力，单个进程也无法在内部直接获得多核并行能力；
- 若改用多个进程来表达并发，又必须为每个进程创建独立的虚拟地址空间、页表和上下文，带来沉重的隔离开销与昂贵的 IPC 成本。

线程（Thread）的真正创造不是“发明一个更小的进程”，而是：

把“资源所有权”与“独立执行路径”彻底解耦

因此，一个现代进程的精确形式化定义为：

Process Instance = Identity/Lifecycle Scope (G) + Shared Resource/Protection Context (R) + {Independent Execution}

一句真言定乾坤

进程回答“我们共同生活在哪个世界”；线程回答“我现在独立执行到哪里”。

3.3 独立推进必然推出 PC、寄存器与栈的独立私有

线程私有 PC、寄存器和调用栈不是教材的主观规定，而是从第一性原理推导出来的逻辑必然：

Independent Progress (独立推进) $\implies$ Independent Resumable State (独立可恢复现场)

推理链条极度严密：

- **独立 PC**：若多条执行流共用同一个 PC，它们根本不可能同时停在两个不同的指令执行位置；
- **独立通用寄存器**：若共用寄存器，执行流 A 的局部运算临时值会被执行流 B 的指令无情覆盖；
- **独立 SP 与调用栈**：若共用同一个栈指针与栈空间，函数调用的返回地址（Return Address）、局部变量和栈帧链会彻底互相践踏、完全崩溃；
- **独立调度状态**：若共用状态，系统就无法单独让执行流 U_1 处于 Running，而让执行流 U_2 处于 Blocked。

因此，线程的第一本质不是“被调度”，而是“可独立保存与恢复的执行上下文”

Execution Context (U_i) = ($PC_i, Registers_i, SP_i, Stack_i, ExecutionState_i, TLS_i$)

3.4 共享与私有：三权分立模型 (Ownership $\neq$ Addressability $\neq$ Concurrency Safety)

在考察“线程共享什么、私有什么”时，传统教学最容易因为一句“栈私有”引发认知混乱。必须严格区分分解三个不同维度：

Ownership (语义归属) $\neq$ Addressability (硬件寻址能力) $\neq$ Concurrency Safety (并发正确性)

1. **Ownership (所有权 / 语义归属)**：这份状态在程序逻辑上属于谁？谁的生命周期与函数调用栈需要维护它？

- 栈帧、SP、局部变量、PC 寄存器语义上属于特定线程 U_A ；
- 考试问“栈属于谁”，标准答案是：**每条线程独立私有**。

2. **Addressability (可寻址性 / 硬件保护边界)**：同进程的其他线程在硬件 MMU 和虚拟地址空间层面，是否有能力访问这块内存？

- 同一进程的所有线程共享同一个虚拟地址空间（同一份页表基址 CR3）；
- 硬件 MMU 的保护颗粒是“页”（Page），它只认进程的地址空间边界，不认线程边界；
- 若线程 A 将自己栈上局部变量的指针 $\&x$ 传递给线程 B，线程 B 可以毫无阻碍地直接通过指针解引用 $*ptr$ 读取或修改线程 A 栈中的数据！
- 结论：**Per-thread Ownership $\nRightarrow$ Address-space Isolation**（线程级所有权不等于硬件内存隔离）。

3. **Concurrency Safety (并发安全性)**：能访问不等于应该随意并发访问。两条执行流都能访问同一内存地址，若缺乏同步机制，就会产生数据竞态（Data Race）。因此：

Shared Mutable State + Uncontrolled Interleaving $\longrightarrow$ Race Condition

这正是为什么线程专题在此停止，而将互斥、锁与同步控制交由 OS-03 专题完整拥有。

3.5 线程局部存储 (TLS)：同一地址空间中的逻辑私有状态

线程局部存储（Thread Local Storage, TLS）是“逻辑私有 $\neq$ 地址隔离”的最佳工程证据：

- 尽管所有线程共享同一个全局虚拟地址空间，运行时（Runtime）仍可为每个线程在同一地址空间中开辟专属区域，使每个线程访问同一变量名 x 时，实际解析到各自独立的实例：

$$U_1 \rightarrow x_1, \quad U_2 \rightarrow x_2, \quad U_3 \rightarrow x_3$$

- 典型案例：C 标准库的全局错误码 `errno`。在单线程时代是全局整数；在多线程时代被重定义为 TLS 宏，确保线程 A 的系统调用失败不会覆盖线程 B 的 `errno`。

TLS 的工程实现窗口

在现代 x86-64 架构下，TLS 通常由专门的段寄存器基址（如 `%fs` 寄存器）指向当前线程的线程控制块（TCB/pthread 结构）与动态线程向量（DTV），通过相对于 `%fs:0` 的偏移量寻址；在 ARM64 下则使用 `TPIDR_ELO` 寄存器。408 无需记忆寄存器名字，只需守住核心结论：**TLS 在不破坏共享地址空间的前提下，通过基址指针偏移提供了线程局部数据抽象。**

3.6 共享文件描述符：共享句柄意味着共享下游可变引用状态

多线程不仅共享“文件描述符表”本身，更关键的是共享了文件系统中的打开状态（Open File Description）：

Thread $U_1, U_2 \xrightarrow{\text{share}} fd \xrightarrow{\text{reference}} \text{Open File Description (f_pos, f_flags)} \xrightarrow{\text{reference}} \text{Inode / File Object}$

若线程 U_1 执行 `read(fd, buf, 100)`，底层的读取偏移量指针 `f_pos` 会向前移动 100 字节；紧接着线程 U_2 执行 `read(fd, buf, 100)`，将直接从第 101 字节开始读取！**共享 Handle 等价于共享下游可变状态的引用**，这是跨越“进程/线程”与“文件系统/I/O”的核心桥梁。

3.7 并发、并行与 408 进程/线程全维度对照

- **Concurrency (并发)**: 多条逻辑执行流在同一时间段内交替向前推进（单核或多核皆可）；
- **Parallelism (并行)**: 多条执行流在同一物理时刻于不同 CPU 核心上物理同时执行（必须依赖多核/多处理器）。

对比维度	进程 (Process)	同一进程内的线程 (Thread)
核心角色	资源分配、保护边界与运行容器	独立执行路径与 CPU 调度基本实体
地址空间	彼此独立隔离（独立页表 CR3）	共享所属进程的整个虚拟地址空间
执行现场	至少拥有一个主执行上下文	每条线程拥有独立私有的 PC、通用寄存器、SP 与调用栈
局部私有存储	独立的数据段、堆与栈	每线程独立栈、寄存器现场与 TLS（如 <code>errno</code> ）
切换开销	高：切换特权级、内核栈、CR3 页表，并承受 TLB 与 Cache 冷失效	低：同进程切换通常仅保存/恢复寄存器与 SP，无需切换页表
通信机制	低速：需经由 IPC（管道、消息队列、共享内存映射）	高速：直接读写共享堆/全局变量（需同步原语保证安全）
故障隔离性	强：单个进程崩溃通常不直接波及其他进程	弱：共享内存空间，单个线程段错误通常导致全进程崩溃
生命周期管理	创建开销大（ <code>fork</code> 分配 PCB 与页表）； <code>exit</code> 留 zombie	创建轻量（ <code>pthread_create</code> 分配栈与 TCB）； <code>pthread_join</code> 回收

4 PCB：把“进程是什么”落实到内核可管理的数据结构

4.1 PCB 不是进程本体，而是内核管理进程的控制记录

408 中可以把进程控制块（Process Control Block, PCB）理解为：

PCB = Identity + Processor State + Scheduling/Control + Resource References

它解决的问题不是“程序的数据放在哪里”，而是：内核怎样识别这个进程、暂停它、重新调度它，并找到它所拥有或引用的资源。

PCB 信息类别	典型字段 / 信息	解决的问题
进程描述信息	PID、父进程关系、用户/权限标识等	“它是谁，属于谁？”
处理机状态信息	PC、SP、PSW/状态寄存器、通用寄存器等需要保存的执行现场	“下次从哪里、以什么 CPU 状态继续？”
调度与控制信息	当前状态、优先级、时间片/调度参数、队列链接、等待原因等	“现在能不能运行，排在哪里？”
资源与管理信息	地址空间/页表相关引用、打开资源引用、I/O 或其他内核对象引用等	“它依附于哪些资源环境？”

最容易混淆的两层

PCB 保存的是管理信息与资源引用，不等于把进程的代码、堆、栈、文件内容都“装进 PCB”。地址空间、文件对象等由各自子系统维护，PCB/任务结构只持有能找到这些对象的状态或引用。

“PCB 是进程存在的唯一标志” 应该怎样理解

408 教材常说 PCB 是进程存在的唯一标志。这句话的稳定语义不是“PCB 就等于进程”，而是：只有 OS 为该运行实例保留了可定位的内核控制记录，它才作为一个可被识别、排队、等待、统计和回收的进程存在于 OS 的管理世界中。因此 PCB 属于内核管理状态，用户进程不能把它当普通用户内存任意改写。

创建进程也不能粗暴压成“只分配一个 PCB”：从管理身份看，建立控制记录是关键锚点；但一个真正可运行的新实例还必须建立/继承必要的地址空间关系、执行入口、资源引用和队列状态。这里再次体现：

Management Anchor ≠ Whole Process Object

4.2 PCB 的组织方式：本质是“怎样按身份/状态快速找到控制记录”

408 传统教材还会问 PCB 的线性、链接、索引组织。不要把它们当三个新的进程模型，它们只是同一批控制记录的三种索引策略：

组织方式	核心表示	主要权衡
线性	PCB 放在表中，按条件扫描	结构简单，但按状态寻找要扫描更多记录
链接	PCB 按 Ready、不同 Wait Event、Free 等关系串成队列/链	状态/事件队列天然对应本册 Q/W 模型，插入删除方便；需维护链接关系
索引	为不同状态/类别维护 PCB 地址索引表	查找集中，但索引本身也需要在状态迁移时同步维护

真正的不变量是：内核必须能按 identity 找到对象，并能按 scheduling/wait state 找到候选集合；状态改变时，控制记录与相应索引/队列必须一致地改变。所以“PCB 组织方式”应挂在 Objects/Relations/Queues，而不是另起一套死记表。

4.3 PCB 与线程上下文：教材抽象和现代实现要分层

单线程进程的教材模型常把“进程执行现场”一起归入 PCB；引入多线程后，更清楚的拆法是：

Process Resource Context + per-thread Execution Context

每条线程必须有独立 PC、寄存器、栈和调度状态；共享地址空间、文件等进程级资源。工程实现中这些信息可能分散在 task/thread/process 等多个内核结构中，408 做题先守住语义，不要把某个具体内核结构名反过来当定义。

4.4 TCB：把线程自己的执行状态单独拿出来

408 教材常用线程控制块（Thread Control Block, TCB）表示一条线程对应的控制记录。最稳妥的语义模型是：

TCB = Thread Identity + Execution Context + Scheduling State + Thread-local Metadata

判断问题	PCB / 进程级状态	TCB / 线程级状态
“属于谁？”	PID、进程身份与权限、父子关系等	TID、线程身份、所属进程关系等
“资源世界是什么？”	地址空间、打开资源等进程级上下文或其引用	通常不再复制一套独立进程资源世界
“下一条从哪执行？”	单线程教材模型可把执行现场并入 PCB	每线程独立的 PC、SP、寄存器与栈
“现在能否获得 CPU？”	进程级教材模型可记录调度状态	内核可见线程模型下，每线程有自己的 Ready/Running/Blocked 等调度状态

PCB / TCB 的判定口令

先问题目描述的是**资源与保护环境**，还是一条**独立执行路径的现场**。前者归到进程级上下文；后者归到线程级上下文。不要靠“字段名长得像 PCB 还是 TCB”猜答案。

TCB 是教材抽象，不是跨系统统一的结构体名字

不同 OS、线程库和运行时可能把线程状态拆进不同结构。考试中使用 PCB/TCB 术语没有问题；进入工程层后，应追踪“谁保存线程身份、寄存器现场、栈与调度状态”，而不是要求源码中一定存在一个名为 TCB 的独立对象。

5 进程通信 IPC：隔离之后怎样交换信息

5.1 母问题：进程彼此隔离，为什么还能通信？

进程模型首先承诺隔离：一个进程不能默认把另一个进程的普通私有地址直接当作自己的内存读写。现实程序又必须协作，因此 OS 必须提供**受控的跨进程信息通道**：

Isolation → Communication Need → { Shared State
Kernel-mediated Transfer / Notification

这比背“管道、消息队列、共享内存、信号”更重要：先判断**跨进程传递的究竟是共享状态、数据流/消息，还是事件通知**，再判断对应对象、队列和阻塞条件。

IPC 不要先背名字：先检查 Channel State

从这批标准 408 笔记中最值得吸收的不是更多 IPC 名词，而是把通信通道也当成一个有状态对象：

Channel State = (Payload Type, Capacity, Endpoint Liveness, Wait Conditions)

先问四件事：传的是**共享状态、数据/消息还是事件**？通道能不能暂存以及是否已满/空？另一端是否仍然存在？当前 send/read/receive 缺的条件是什么？很多“会不会阻塞”的结论都由这四项直接生成，而不是由 IPC 名字单独决定。

5.2 四类典型 IPC：看“共享什么、等什么”

IPC 形式	共享/传递的核心对象	典型等待条件	主要机制与代价
共享内存	双方映射到同一共享内存对象/物理实现	数据本身不会自动产生“可读事件”；同步条件由程序协议定义	数据访问路径短，但共享写会产生 race，需要 OS-03 的同步机制；映射细节归 OS-04
管道 (Pipe)	内核维护的有序字节流与缓冲状态	读端遇到“当前无数据但可能有后续写入”；写端也可能因容量/语义受限而等待	通过内核对象连接读写端，天然具有 stream、buffer、block/wakeup 语义
消息传递	内核/运行时维护的 message / mailbox / queue	无可接收消息、队列容量不足或同步发送尚未匹配接收方	边界清楚，便于隔离；通常需要排队、复制或引用管理
信号/事件通知	“某事件发生”的控制信息	通常不是大块数据传输通道	信息量小但适合通知、唤醒或控制；具体递送语义依机制而异

IPC 题第一动作：先画对象，再画状态

不要先问“这是哪一种 IPC 名字”。先写：**通信对象在哪里？当前有什么状态？哪一方在等待什么条件？哪个事件会改变条件？**例如匿名管道的进程侧轨迹可以压成：

$$\text{read(pipe)} + \text{No Data} \rightarrow \text{Blocked} \xrightarrow{\text{writer produces data / closes endpoint}} \text{Ready}.$$

注意最后通常只是 **Ready**，并不保证读取者在事件发生瞬间立刻 **Running**。

用 Channel State 直接推出三类高频边界

- **有界消息/管道已满**：若发送语义要求等待空间，则 writer/sender 缺的是 Capacity，可能进入等待；不是“发送这个动作天然阻塞”。
- **管道无数据但仍有写端存在**：reader 等的是“未来数据或 endpoint 状态变化”；若所有写端都关闭且缓冲已读空，条件已经变成 **EOF**，此时应返回结束而不是永久 Blocked。
- **所有读端都关闭**：writer 面对的是“已无合法接收端”，应得到错误/异常语义，而不是把它当作“缓冲区暂时满”继续等。

这三条统一说明：**阻塞判定必须读取 Capacity + Endpoint Liveness + Operation Semantics。**

408 信号模型：Pending、Mask、Disposition 三件事分开

经典教材常把普通信号抽象成“待处理位向量 + 屏蔽位向量”。更稳定的状态模型是：

$$\text{Signal State} = \text{Pending} + \text{Mask} + \text{Disposition}$$

Pending 回答“事件已到但尚未处理吗”，Mask 回答“当前线程是否暂缓递交”，Disposition 回答“递交后忽略、默认处理还是执行 handler”。经典普通信号的位集合抽象不能用来统计同种事件发生了多少次；工程上还存在可排队的实时信号，因此不要把“所有 signal 永远不排队”升级成跨系统绝对结论。

5.3 共享内存为什么“快”，又为什么不等于“简单”

共享内存把主要数据访问变成普通 load/store，避免每次逻辑通信都必须经过“发送一份消息 → 内核缓冲 → 接收一份消息”的路径。但它只解决**双方怎样看见同一份状态**，并没有自动解决：

- 两个执行流同时修改时谁先谁后；
- 一个线程/进程何时可以认为数据已经准备完成；
- 是否存在 race、lost update、可见性或一致性问题。

因此：

$$\text{Shared Memory IPC} \neq \text{Synchronization}$$

共享映射的建立与地址空间机制由 OS-04 解释；互斥、条件同步与 PV 由 OS-03 解释。本册只拥有“**进程为何需要通信、通信对象如何影响 Blocked/Ready 状态**”这一层。

IPC 跨 Topic 停止规则

若题目问“两个进程怎样交换信息、谁会阻塞、什么事件唤醒”，留在本册；若继续追问共享页如何映射，转 OS-04；追问 mutex/PV/条件变量，转 OS-03；追问 fd/OFD/inode 生命周期，转 OS-06/07；追问 TCP/UDP socket 的运输层端点语义，转 Network。**不要让 IPC 变成复制四个专题正文的借口。**

6 进程资源接口：本册只追踪“引用怎样随生命周期交接”

进程确实需要保存打开文件等资源的引用，但根据本项目的 Topic 边界，本册只追到：

$$\text{Process} \rightarrow \text{fd table} \rightarrow \text{File-System Open State}$$

后一层的 OFD、offset、dentry/inode 与文件生命周期由文件系统专题完整解释。

这里仅保留与进程生命周期直接相关的三条接口语义：

- **fork()**：子进程获得自己的进程管理状态与 fd 表语义副本；对应项可继续引用同一打开实例，因此资源可以跨父子进程共享；
- **exec()**：替换程序映像，不等于创建新进程；未被 close-on-exec 规则关闭的 fd 可以继续存在；

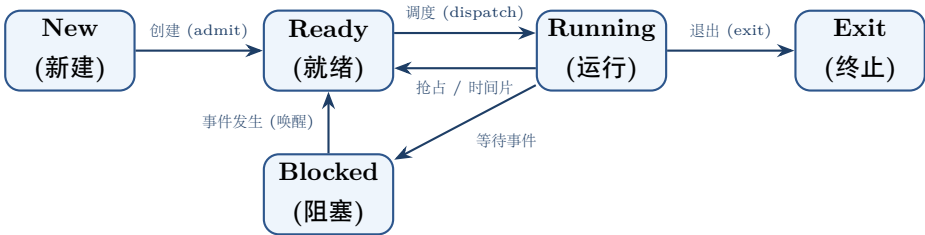
- `exit()`：结束进程执行并释放其持有的运行时引用；“文件对象何时真正消失”属于文件系统生命周期问题。

跨 Topic 时的停止规则

当题目只问进程状态、调度或 `fork/exec`，追到“资源引用被继承/保留/释放”即可；只有题目继续追问 `offset`、硬链接、`inode`、`unlink` 或崩溃一致性时，才进入 OS-06/07 的文件系统模型。

7 进程生命周期主线

7.1 生命周期总览



状态转换的第一原则

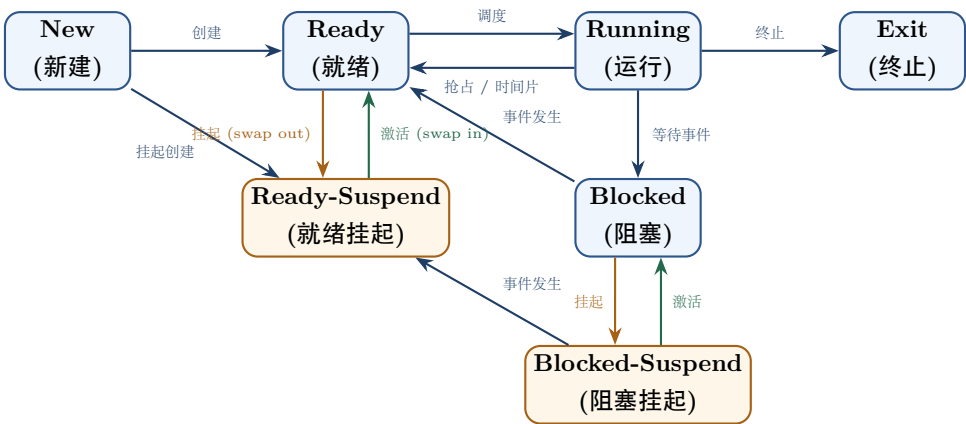
每条状态箭头都必须有事件依据。如果说不出事件，就不应随意改变状态。

7.2 五态为什么还不够：挂起与七态模型

五态模型回答“任务能不能运行、是否正在运行”；当系统还要表达暂时移出内存/暂停参与调度时，需要再增加 Suspend 维度：

Runnable/Blocked 状态 × Resident/Suspended 状态

扩展后的完整七态模型（包含就绪挂起与阻塞挂起）结构如下：



关键转换按“事件是否满足 + 是否重新激活”推：

状态转换	触发事件	转换原因与含义
Ready → Ready-Suspend	suspend / swap out	本来能运行，但暂时被调出内存
Blocked → Blocked-Suspend	suspend / swap out	等待事件的同时被调出内存
Blocked-Suspend → Ready-Suspend	所等事件发生	阻塞原因消失，但尚未激活回内存
Ready-Suspend → Ready	activate / swap in	已满足运行条件且重新调入内存就绪队列
Blocked-Suspend → Blocked	activate (事件未发生)	调回内存后仍然缺少等待条件

Suspend 与 Blocked 不是一回事

Blocked 的判据是“即使现在给 CPU 也无法继续”；Suspend 的判据是“当前被系统暂停/移出正常可调度驻留集合”。因此事件可以让 Blocked-Suspend 先变成 Ready-Suspend，却不会凭空让它直接 Running。

七态模型与后文**中级调度**是同一件事的状态侧与机制侧：状态图告诉你“对象去了哪里”，中级调度告诉你“谁负责做这次挂起/激活”。

7.3 状态不是一个标签：它必须落实为 Queue Membership + Wait Relation

标准 408 笔记常讲“PCB 按状态组织成就绪队列、若干阻塞队列”。这不是实现细节，而正好落在本书的 OS 母模型：

$$\text{State Change} \implies \text{Relation / Queue Membership Change}$$

例如一次真正的阻塞不能只写成 `state=Blocked`，它至少意味着：

Running on CPU → save resumable state → record WaitFor(E) → W_E → schedule another runnable entity.

而事件 E 发生时：

$$W_E \xrightarrow{E} \text{Ready Queue}$$

因此**按阻塞原因维护不同等待队列**的深层意义不是“教材规定很多链表”，而是让事件发生时直接定位“谁正在等这个事件”，把 Wakeup(E) 从全局扫描变成针对相应等待集合的状态迁移。

五态图三条禁边，用机制而不是背图判断

- **Ready** $\nrightarrow$ **Blocked**: Ready 实体没有执行指令，因而不可能此刻主动发出新的阻塞请求；
- **Blocked** $\nrightarrow$ **Running**: 事件完成只消除了等待条件，使其重新具备运行资格；真正获得 CPU 还缺 scheduler/dispatch；
- **Running** → **Ready**: 必须存在抢占、时间片耗尽或其他“CPU 被收回”的机制；在经典非抢占式调度下不会无缘无故发生；
- **Terminated** $\nrightarrow$ **Runnable**: 生命周期已经结束，重新运行意味着创建另一个实例，而不是复活原进程。

7.4 进程控制原语：一次合法状态变化是原子状态事务

创建、终止、阻塞、唤醒不要背成四段伪代码。它们共同承担一件事：把多个相互关联的内核状态从一个合法截面一次性迁移到另一个合法截面。

Primitive = Atomic Transition over (G, R, U/K, Q, W)

例如 Block 必须让“当前执行资格、保存现场、等待原因、队列成员关系、CPU 控制权”彼此一致；若只完成一半就被观察到，可能出现“既不在运行队列也不在等待队列”的坏状态。

经典教材常用“关中断”解释单处理机上的不可分割性，但应把原子语义与具体实现机制分开：

- 单 CPU 教材模型中，屏蔽本地中断可以避免当前处理器被普通抢占打断；
- 多处理机上，一个 CPU 关中断并不能阻止另一个 CPU 同时修改共享内核状态，因此还需要锁、硬件原子指令等跨 CPU 同步机制；
- 所以真正稳定的不变量是“状态迁移不可暴露半完成中间态”，不是“所有原语都等价于关中断”。

四类原语可以压成：

原语	改变的核心对象/关系	结束后必须成立
Create	建立新的 G、必要的 R 绑定与初始 U/K，加入可管理集合/队列	新实例有唯一身份且处于合法初态
Block	当前执行实体 $\rightarrow W_E$ ，释放 CPU 承载	等待原因明确，不能继续被当作 runnable
Wakeup	$W_E \rightarrow Q$	等待条件已解除，但尚不保证 Running
Terminate	终止执行流/进程生命周期、撤销队列与资源引用并保留必要的可观察退出状态	终止实体不再 runnable；尚待观察的退出信息仍可定位

7.5 阶段 A：创建——从父进程产生新执行环境

Unix/POSIX 语义中，fork() 创建子进程。关键不是“复制所有物理内存”，而是创建一个新的进程语义实体，并让其初始状态看起来接近父进程。

fork() 成功后，父子进程都会从“这次调用返回之后”继续执行，但返回值不同：

parent: return child PID child: return 0

因此代码中的 if (fork()==0) 本质是在用返回值区分当前这条执行流到底属于父还是子。

连续 fork 题不要机械写 2^n

第一动作是沿控制流逐次问：当前有多少条执行流真正执行到了这一条 fork？每一个成功执行到的 fork() 才把对应执行流复制成父、子两条。若前面存在 if/else、exit()、循环或某些分支只有父/子能进入，就不能把源码中出现的 fork 个数直接代入 2^n 。

现代实现常通过 COW 优化：

初始共享物理页 + 只读保护 $\xrightarrow{\text{首次写}}$ 复制对应页

文件资源采用另一套语义：父子有各自的 fd 表语义副本，但对应条目可以继续引用同一个文件系统打开实例。这里追踪到“引用关系被继承”为止，打开实例内部状态交给文件系统专题。

7.6 阶段 B: exec——不是新建进程，而是替换进程映像

exec() 最应该记住一句：

same process identity, new program image

典型结果：

- 进程 PID 不因为 exec 本身变成另一个新 PID；
- 原有代码、数据、栈等程序映像被新的可执行程序替换；
- 未设置 close-on-exec 的打开文件描述符可以跨 exec 保留。

7.7 阶段 C: Ready——只缺 CPU

Ready 的精确定义不是“正在等待某事”，而是：

运行所需条件已满足，只差 CPU 执行机会

因此就绪队列中的实体如果此刻拿到 CPU，就能继续执行。

7.8 阶段 D: Running——用户态和内核态都可能属于“这个进程正在运行”

当进程执行系统调用进入内核时，常常仍然是当前进程上下文。

因此：

Running ≠ User Mode

进程可以在内核中替自己执行系统调用处理，随后返回用户态；也可以在内核中发现条件不满足并进入阻塞。

7.9 阶段 E: Blocked——即使给 CPU 也不能继续

Blocked 与 Ready 的核心区别：

对比维度	就绪态 (Ready)	阻塞态 (Blocked)
缺少的核心资源	只缺 CPU 时间片	缺少特定的外部事件或资源条件
给 CPU 能否继续运行	能立即投入 Running 运行	不能，即使分配 CPU 也无法向前推进
进入触发原因	被抢占、刚创建完成、从 Blocked 被唤醒	等待磁盘 I/O、信号量 P 操作、子进程退出等
离开触发原因	调度器选择 (Scheduler Pick)	所等待的外部事件/资源就绪

“调用 read 就 Running→Blocked” 也是过度简化

read() 如果数据已经在缓存/缓冲中，可能直接完成并返回。只有在当前语义要求等待且数据尚未可用时，执行实体才可能睡眠/阻塞。类似地，用户态 mutex 的无竞争获取甚至可能不需要系统调用。

7.10 阶段 F: Wakeup——Blocked $\rightarrow$ Ready, 不等于马上 Running

设备完成、中断到来、条件被满足后，内核通常只是把等待实体变成 runnable/ready:

Blocked $\rightarrow$ Ready

它仍然要等待 scheduler 选择:

Ready $\rightarrow$ Running

这是 408 状态题最关键的两步分离。

7.11 阶段 G: Exit / Zombie / Wait

进程执行 `exit()` 后，绝大多数运行资源会释放，但 Unix 风格系统通常保留少量退出状态，供父进程通过 `wait()` 回收，因此出现 zombie 概念。

应理解为:

Process Execution Ends $\neq$ All Kernel Metadata Immediately Vanishes

Zombie $\neq$ Orphan: 两个词根本不在同一维度

Zombie 问“child 是否已经终止但退出状态尚未被父进程消费/reap”; **Orphan** 问“child 仍存在时，它原来的 parent 是否已经先结束，从而父子关系需要被重新接管”。前者是**终止状态的观察/回收问题**，后者是**进程关系变化问题**。判断时先问“谁先结束”，再问 child 此刻是否仍能执行。

7.12 线程生命周期、线程组 (G) 与事件作用域

当系统引入多线程后，生命周期与异步事件不再仅仅是单条流的线性演进，而必须明确区分事件的作用域 (**Event Scope**):

Event Scope:	Thread-Level (U_i)	$\longleftrightarrow$	Thread Group / Process-Level (G)	$\longleftrightarrow$	System-Level
--------------	------------------------	-----------------------	--------------------------------------	-----------------------	--------------

1. 线程组 (G) 与信号路由 (Signal Routing):

- **线程定向信号 (Thread-directed)**: 由当前执行流同步引发的硬件异常（如除零 SIGFPE、非法内存访问 SIGSEGV），由硬件与内核直接定向派发给触发它的具体线程 U_i ;
- **进程定向信号 (Process-directed)**: 外部异步产生的事件（如终端中断 SIGINT、终止信号 SIGTERM），发送给整个线程组 G 。内核在所有未屏蔽该信号的线程中挑选一条执行信号处理程序 (Signal Handler);
- **共享处置与独立屏蔽**: 信号处理函数表 (Signal Dispositions) 全进程共享；而每条线程拥有独立的信号屏蔽字 (Signal Mask)。

2. 退出粒度: 单线程退出 vs 整个线程组退出:

- 单条线程调用 `pthread_exit()`: 仅终止当前执行上下文 U_i ，其所持有的私有栈和寄存器现场被回收，同进程其他线程继续正常运行;

- 任意线程调用 `exit()` 或 `exit_group()`，或主线程直接从 `main()` 返回：触发进程级终止，整个线程组 G 内的所有执行流全部被强制终止。

3. `pthread_create()` 的抽象语义：创建线程不是普通的函数调用，而是在系统中增加一条全新的独立执行流：

$$S \xrightarrow{\text{create}} S' \quad (\text{分配新栈帧、初始化寄存器现场与} TLS, \text{置} U_{\text{new}} \in Q_U \text{ 进入 Ready})$$

4. `pthread_join()` 的抽象语义：生命周期状态同步：`join(T)` 根本不是数据通信 (IPC)，而是纯粹的生命周期终止等待：

join(T) ≡ WaitUntil(State(T) = Terminated)

- 线程 U_A 调用 `pthread_join( $U_B$ )`： U_A 从 `Running` $\rightarrow$ `Blocked`，加入 U_B 的终止等待队列；
- U_B 完成计算并调用 `pthread_exit()`： $U_B \rightarrow$ `Terminated`，释放其底层栈与资源，内核/运行时触发 `Wakeup( $U_A$ )`；
- U_A 重新变为 `Ready`，随后被调度重新进入 `Running`。

8 调度：控制权已经回到内核，下一步给谁？

8.1 先分清三级调度：它们选择的不是同一种“下一位”

“调度”不是只有 `ready queue` 里挑一个进程。408 经典模型按决策层次分为：

层次	又称	典型状态/位置变化	决策问题
高级调度	作业调度	后备作业 $\rightarrow$ 被接纳进入系统/内存并建立进程	“哪些作业现在可以进入执行系统？”
中级调度	内存调度 / 对换相关调度	<code>Ready/Blocked</code> $\leftrightarrow$ <code>Ready-Suspend/Blocked-Suspend</code>	“哪些进程暂时挂起，哪些重新激活？”
低级调度	进程调度 / CPU 调度	<code>Ready</code> $\rightarrow$ <code>Running</code>	“当前 CPU 下一步运行谁？”

一眼定位

看到“后备队列/作业接纳”先想**高级调度**；看到“挂起、激活、对换”先想**中级调度**；看到“就绪队列、时间片、CPU 分配”先想**低级调度**。本册后续 FCFS/SJF/RR 等算法主要讨论低级 CPU 调度。

教材模型与现代系统边界

三级调度是 408 的经典抽象。现代通用操作系统未必存在传统批处理意义上的“作业调度”，挂起/换出机制也不会机械等同于整进程 `swap`。考试先按题设和教材模型作答，再区分工程实现。

8.2 Scheduler $\neq$ Dispatcher：先决定，再真正交出 CPU

标准教材把“调度实现”继续拆成排队、决策与交权，这个区分非常值得进入母模型：

Event makes task runnable $\rightarrow$ Enqueue $\rightarrow$ Scheduler Pick $\rightarrow$ Dispatcher / Context Switch $\rightarrow$ Running

决定谁把 CPU 真正交给它

- **Queueing / 排队**: 维护 runnable 集合及其顺序/优先级结构;
- **Scheduler / 调度程序**: 根据 policy 从合法候选中选择下一执行实体;
- **Dispatcher / 分派程序**: 执行这个决定, 完成必要的现场/地址空间/模式切换并把控制流恢复到被选实体;
- **Dispatch Latency / 分派延迟**: 从“已经选中”到“新实体真正开始执行”的纯交接时间。

因此:

$$\text{Scheduling Decision} \neq \text{Context Switch}$$

调度器可能再次选中当前实体, 此时发生了一次 decision, 却不必真正换人; 反过来, 真正切到另一个执行实体之前必然已经存在一个选择/交权依据。

8.3 机制与策略必须拆开

机制 / 策略

机制 (mechanism): OS 有没有能力停止当前执行流、保存状态、切换到另一个执行流?

策略 (policy): 面对多个 runnable 实体, 应该选择谁?

例如:

$$\text{Timer Interrupt} + \text{Context Switch Machinery} = \text{Mechanism}$$

而:

$$\text{FCFS/SJF/RR/Priority/MLFQ} = \text{Policy}$$

8.4 调度评价指标是互相冲突的目标函数

先把 408 计算题的四个时间量锁成同一套符号。设任务到达/提交时刻为 A , 首次获得 CPU 时刻为 F , 完成时刻为 C , 实际需要的 CPU 服务时间为 S :

$$T_{\text{turnaround}} = C - A, \quad T_{\text{weighted}} = \frac{C - A}{S}, \quad T_{\text{waiting}} = (C - A) - S, \quad T_{\text{response}} = F - A$$

其中 waiting 用“周转减服务”最稳, 因为抢占式调度下任务可能多次回到 Ready, 不能只拿“首次上 CPU 减到达”代替全部等待。还应保留两个快速校验:

$$T_{\text{weighted}} \geq 1, \quad \overline{T_{\text{weighted}}} = \frac{1}{n} \sum_i \frac{T_i}{S_i} \neq \frac{\sum_i T_i}{\sum_i S_i} \quad (\text{一般情形}).$$

除此以外还要看:

- CPU utilization: 尽量别让 CPU 闲;
- Throughput: 单位时间完成更多工作;
- Fairness: 谁不应长期得不到 CPU, 以及“公平”的对象到底是谁;
- Predictability / deadline: 实时系统特别关注可预测边界而不是只看平均值。

不存在对所有工作负载同时最优的单一调度策略。

8.5 先识别工作负载，再决定“什么叫好”

调度题不应从算法名字开始，而应先确定系统最想保护的目标：

Workload / System Goal → Objective → Scheduling Policy

典型系统/工作负载	首要关注目标	看到题目时的第一判断
批处理 (Batch)	Throughput、Turnaround，尽量高效完成一批作业	更关心单位时间完成多少、平均多久完成，而不是每个任务立刻得到一次响应
分时/交互式 (Time-sharing / Interactive)	Response、Fairness，同时控制切换开销	用户不能长时间感觉“系统没理我”，因此要让 runnable 任务周期性获得执行机会
实时 (Real-time)	Deadline、Predictability / bounded latency	“平均很快”不等于合格；关键是关键任务能否在规定时间内边界内完成或响应

调度算法选择题的第一动作

先圈题干中的吞吐/周转、首次响应/交互、公平、截止时间等目标词，再判断算法的偏好是否与目标一致。不要看到“现代系统”就条件反射选 RR/MLFQ，也不要看到“平均等待时间”就忽略题目给出的抢占条件与 burst 信息。

8.6 经典算法应按“偏好谁”来记

算法	核心偏好	主要优点	核心代价 / 潜在风险
FCFS	先到先服务	实现简单、低调度系统开销	Convoy Effect（护航效应），短任务久等
SJF	短 CPU Burst 优先	理论上可获得最小平均等待时间	难准确预测下一步 Burst，长任务易饥饿
SRTF	剩余时间短者优先	抢占式偏向短任务，降低响应时间	频繁抢占与上下文切换开销大
HRRN	当前响应比最高者优先	同时考虑等待时间与服务时间，长任务等待越久竞争力越强	非抢占式；仍需知道/估计服务时间，计算开销高于 FCFS
Priority	优先级高者优先	直观表达任务重要程度	低优先级任务长时期饥饿（需 Aging 机制）
RR	人人轮流切时间片	响应极佳、保证公平性	时间片 q 过小导致频繁切换；过大退化为 FCFS
MLFQ	用历史行为推断类型	兼顾短交互任务与长计算任务	规则复杂，需要防范 Task Gaming 与饥饿

8.7 MLQ 与 MLFQ：把一条就绪队列拆开后，会新增哪三个决策？

经典 408 调度体系不仅包含 MLFQ，也要求识别多级队列（MLQ）。更重要的是，MLQ 可以直接从“就绪队列结构发生变化”推导，而不必再背一套孤立定义。

单一 Q_K 被拆成：

$$Q_K \longrightarrow \{Q_K^{(1)}, Q_K^{(2)}, \dots, Q_K^{(m)}\}$$

以后，原来一句“从就绪队列选谁”会裂成三个独立问题：

1. **Classification**: 任务进入哪条队列?
2. **Inter-queue Scheduling**: 多条非空队列之间先服务哪条，按固定优先级还是份额/时间片?
3. **Intra-queue Scheduling**: 同一队列内部按 FCFS、RR、Priority 还是其他规则选人?

MLQ 与 MLFQ 的稳定分界恰好落在 Relation/Transition 上：

MLQ: Queue Membership fixed	≠	MLFQ: Queue Membership can change by feedback
-----------------------------	---	---

因此 MLQ 必须在进入系统时就有足够信息完成分类；MLFQ 则允许根据实际 CPU 使用行为重新分类。不要把“有多条队列”直接等同于 MLFQ。

8.8 HRRN：不要只背公式，要看“等待如何改变优先级”

高响应比优先（Highest Response Ratio Next, HRRN）在每次**非抢占式**选择时计算：

$$R = \frac{W + S}{S} = 1 + \frac{W}{S}$$

其中 W 是已经等待的时间， S 是要求服务时间。选择当前 R 最大的作业/进程。

这个式子可以直接从两个极端理解：

- 若等待时间相近， S 小的短任务响应比更高，保留了 SJF 偏爱短任务的倾向；
- 同一任务等待越久， W 持续增大，响应比随之上升，因此经典模型下能显著缓解长任务饥饿。

HRRN 计算题第一动作

不要在所有时刻连续重算。只在 CPU 需要重新选择下一个任务的决策点，对当时已经到达且尚未完成的候选者计算 W 与 R ；一旦选中，因为 HRRN 是非抢占式，该任务运行到本次服务结束。

教材模型 vs 现代 Linux 调度算法

MLFQ 是 408 考纲常考的经典教学策略；现代 Linux 普通任务调度长期采用 CFS 的“公平 CPU”模型，当前内核正在向 EEVDF 方向演进。备考时需以经典 MLFQ 规则为准，理解现代算法对公平性的优化。

8.9 公平性必须带主语：Process Fairness ≠ User Fairness ≠ Thread Fairness

“公平”不能只写成一个形容词。它至少需要：

Fairness = Subject + Entitlement/Share + Accounting + Correction
--

- **对进程公平**：每个 runnable process 被当作一份竞争主体；一个用户多开进程可能因此获得更多总份额；
- **对用户公平**：先给用户分 CPU share，再由该用户内部进程瓜分；多开进程不能改变用户总份额；
- **对线程公平**：内核若以 KLT 为调度单位，线程数本身会改变竞争主体数量；M:1 ULT 则可能先由内核给进程/载体一份，再由 runtime 在 Q_U 内二次分配。

经典“保证调度”可以用一个很好的反馈变量理解：

$$\text{Fairness Ratio} = \frac{\text{Actual CPU Received}}{\text{Entitled CPU Share}}.$$

比值小表示系统“欠”该主体的 CPU 份额更多，应提高其获得 CPU 的机会。真正值得留下的不是某一串调度序列，而是：任何公平性命题都必须先说清公平对象和应得份额，否则“公平”没有可验证含义。

8.10 时间片到底解决什么

时间片不是越小越公平就越好：

$$q \downarrow \Rightarrow \text{Response may improve, ContextSwitchOverhead} \uparrow$$
$$q \uparrow \Rightarrow \text{Overhead} \downarrow, RR \rightarrow FCFS$$

因此调度本质上是：

Latency ↔ Throughput ↔ Fairness ↔ Overhead

8.11 多处理机调度：单 CPU 问“谁”，多 CPU 还必须问“放到哪里”

S_{exec} 中的 C 不能只作为图上的 CPU 终点。进入多处理机以后，调度问题由一维：

Who runs next?

升级为二维：

Who runs next? + On which CPU?

这会新增三个互相拉扯的目标：负载均衡、局部性/处理器亲和性、调度数据结构可扩展性。

Q_K 组织	主要优势	必须支付的代价
全局共享 Run Queue	哪个 CPU 空闲就取任务，天然容易做全局负载均衡	多 CPU 竞争同一队列/锁；任务重新运行时不一定回原 CPU，容易损失 Cache/TLB locality
Per-CPU Run Queues	队列访问局部、锁竞争小，容易保持 processor affinity	各核负载可能失衡，必须额外做 push/pull migration 或周期性 rebalance

因此迁移不能只看“另一个核更空闲”，而要比收益与代价：

Migration Benefit ≈ Waiting Time Saved − (Migration Cost + Locality Loss)

若迁移导致 Cache/TLB 重新预热，甚至在 NUMA 中让线程远离其主要物理页，短任务可能还没把迁移成本赚回来就结束了。所以：

Load Balance ↔ Processor/Memory Affinity

这也是为什么“多核越多就一定线性加速”不是调度结论。更高阶的成组调度、专用处理机分配等经典教材方式可以视为不同的 CPU assignment policy；主模型只需守住 Who + Where + Migration Cost。

8.12 有调度原因，不等于“现在这一条指令就能切换”

408 选择题常把三个概念混成一个：

出现调度原因 $\neq$ 允许立即调度 $\neq$ 最终一定换人

某个事件可以先产生 `reschedule` 的需求，但真正的上下文切换必须发生在内核允许切换的安全点。

经典考试语境下，以下阶段通常不能把当前进程随意切走：

1. **正在处理中断/异常的关键现场**：硬件与内核首先必须把当前处理路径维护到可安全返回或可调度的位置；定时器中断可以提出抢占需求，但不等于在中断处理任意一条指令处直接 `context switch`；
2. **正在执行不可抢占的内核临界区**：如果切走会暴露半更新的内核共享状态，就必须先离开该临界区；
3. **正在执行必须保持原子性的原语/关键原子操作**：既然语义要求不可分割，中途自然不能通过普通调度把它拆开。

“处于临界区就绝不能调度”是错误绝对化

这里说的是**不可抢占的内核临界区/原子区间**。普通用户程序进入自己的临界区，并不会因此自动获得“CPU 不可被抢占”的特权；用户临界区的互斥正确性必须自己由同步机制保证。

调度安全点模型

做题时分两步判断：先问“**是否已经出现调度机会/调度请求?**”，再问“**当前是否处在允许 `context switch` 的安全点?**”。只有第二问也成立，才继续判断调度器最终是否选择另一个执行实体。

9 上下文切换：从教材“三层上下文”映射到四维切换模型

9.1 教材说“上下文”时，不要自动缩成“寄存器”

标准 408 教材常把进程上下文分成三层：

- **用户级上下文**：程序/数据、用户栈、共享内存等进程可见的用户空间状态；
- **寄存器上下文**：PC、PSW/状态寄存器、SP、通用寄存器等当前处理器现场；
- **系统级上下文**：PCB、地址空间管理信息、内核栈以及其他 OS 管理状态。

这套三层划分与我们的四维 Switch 并不冲突，它们回答的是两个不同问题：

Three-layer Context asks “what state belongs to the execution instance?”

Four-dimensional Switch asks “which parts actually change in this transition?”

真正切换时，通常只有寄存器现场需要从 CPU 物理寄存器中保存/恢复；用户级上下文和大量系统级上下文本来就分别存在于内存中的对象里，切换更常表现为**改变当前地址空间/当前任务/当前内核栈等引用或硬件选择状态**，而不是把整套“上下文”逐字节搬走。

所以考试看到“进程上下文包括什么”时，按教材三层回答；看到“这一次切换实际换了什么、为什么贵”时，进入下面的四维模型。

9.2 四维切换模型：到底切了什么？

不要再将切换笼统概括为“换 PCB”。在完备的心智模型中，上下文切换应严格解构为四个正交维度：

Switch = (Privilege, Execution, Memory, Mapping)

切换维度	状态变化	核心动作与代价
1. Privilege / Mode Switch	用户态 $\longleftrightarrow$ 内核态	仅改变 CPU 当前特权级与堆栈指针(切入内核栈)， 不更换当前执行流，不更换虚拟地址空间
2. Execution Context Switch	$U_A \rightarrow U_B$ 或 $K_A \rightarrow K_B$	保存旧实体的 PC、通用寄存器、SP 与栈帧；恢复 新实体的寄存器现场，从其断点继续推进
3. Memory Context Switch	$AddressSpace_A \rightarrow AddressSpace_B$	切换页表基地址寄存器 (x86 的 CR3 / ARM 的 TTBR0)；触发 TLB 刷新或 ASID/PCID 标签重 匹配
4. Runtime Mapping Change	$U_i : K_1 \rightarrow K_2$	在 M:N / 虚拟线程运行时中，将就绪的逻辑执行 流重新绑定到另一个可用内核 Carrier 上

9.3 典型并发场景的四维判定矩阵

执行场景	Privilege (模式)	Execution (现场)	Memory (页表)	Mapping (载体)
系统调用直接完成返回	✓	×	×	×
同一 Carrier 上的两个 ULT 切换	×	✓	×	✓
同进程内两个 KLT 切换	✓	✓	×	×
跨进程切换 (Process Switch)	✓	✓	✓	×
Goroutine / Virtual Thread 换核	×	✓	×	✓

9.4 切换开销的深度解构：直接成本与间接成本

线程切换为什么通常比跨进程切换便宜？不要只背“不刷 TLB”，完整成本公式应表达为：

$C_{\text{switch}} = C_{\text{save/restore}} + C_{\text{scheduler}} + C_{\text{privilege}} + C_{\text{memory}} + C_{\text{locality}}$

1. 直接硬件与软件开销 (Direct Costs):

- $C_{\text{save/restore}}$: 通过内存指令保存与加载通用寄存器、浮点寄存器、SP 与 PC;
- $C_{\text{scheduler}}$: 内核或运行时执行调度算法（如遍历红黑树、计算时间片、选择就绪实体）的代码
执行时间;
- $C_{\text{privilege}}$: 模式切换引起的 CPU 流水线冲刷与特权级校验开销。

2. 间接体系结构开销 (Indirect / Locality Costs):

- C_{memory} : 更换页表导致的页表树根节点重置与 MMU 翻译开销;
- C_{locality} (**最大隐形开销**): 跨进程切换后, 新进程的代码与数据与当前 CPU L1/L2/L3 Cache 中的缓存行不匹配, 引发大量的 **Cache Cold Miss (冷失效)**; 同时 TLB 未命中引发昂贵的硬件页表遍历 (Page Walk)。

408 核心考点定性

同进程内的线程切换无需改变页表基址 ($C_{\text{memory}} = 0$), 且由于共享代码段与堆数据, CPU 内部的 Cache 与 TLB 命中率受到的破坏远小于跨进程切换, 因此同进程线程切换开销显著低于跨进程切换。

9.5 动态轨迹: A 切到 B

比“保存 PCB → 恢复 PCB”更清晰的真实执行轨迹是:

$$A_{\text{user}} \xrightarrow{\text{Trap / Interrupt}} A_{\text{kernel}} \xrightarrow{\text{Scheduler Pick}} \text{Switch Context} \rightarrow B_{\text{kernel}} \xrightarrow{\text{Iret / Sysret}} B_{\text{user}}$$

其中调度器与上下文切换代码本身运行在受保护的内核控制路径中。

10 系统调用、中断、异常: 控制流为什么会改变

10.1 中断响应要拆成两层: 硬件自动入口与软件处理程序

408 常把“CPU 响应中断”压成一句话, 容易让学生把所有现场保存都误以为是硬件自动完成。更稳定的分层是:

Hardware Entry → Software Handler → Return/Schedule

1. **硬件自动入口 (中断隐指令的教材抽象)**: CPU 在满足响应条件时建立一个可返回的受保护入口。
408 经典表述通常概括为**关中断/暂时屏蔽同级可屏蔽中断、保存断点与必要状态、根据中断向量引出处理程序**。抽象上至少要保证返回所需的 PC/状态信息不会丢失, 并切入内核控制路径;
2. **软件入口保存现场**: 中断处理程序的入口代码继续保存需要保护的通用寄存器、必要的控制状态/屏蔽信息等。“保存断点”与“保存完整现场”不是同一个动作;
3. **执行中断服务**: 确认中断源、处理设备/时钟事件、更新内核状态, 必要时把等待任务从 Blocked 改为 Ready。若体系结构与内核策略允许中断嵌套, 可以在建立好安全现场后重新允许更高优先级/合适类别的中断;
4. **恢复与返回**: 恢复软件保存的现场, 再通过体系结构规定的中断返回指令恢复硬件入口保存的返回状态。若这次中断产生了抢占需求, 也应在允许调度的安全点先进入 scheduler, 而不是在处理程序任意位置强行切走。

408 高频辨析

- 硬件保存**最小返回状态**，不等于硬件替软件保存了所有通用寄存器；
- “中断到来”只意味着获得一次内核控制机会，不等于一定调度，更不等于一定换进程；
- 经典“关中断 → 保存断点 → 引出 ISR”是教材级抽象；不同体系结构对 PC/PSW、栈切换、屏蔽位的具体自动动作可以不同。

中断、异常与系统调用不要机械套同一条微观顺序

三者都可能进入受保护的内核入口，但来源与恢复语义不同：外部中断是异步事件，异常由当前指令触发，系统调用是程序主动请求。考试若明确问“中断隐指令”，按中断响应模型作答；若问缺页异常或 syscall，则沿各自的 trap/exception 语义推演。

10.2 一次系统调用的五阶段模板

1. 用户代码准备参数；
2. 执行体系结构规定的 trap/syscall 指令；
3. 硬件保存足够的返回状态并进入内核入口；
4. 内核验证参数、执行服务；
5. 若可立即完成则 return；若必须等待，则阻塞当前实体并调度其他实体。

因此系统调用有两条最重要分支：

Fast: syscall → return same task

Slow: syscall → block → schedule someone else

10.3 时钟中断为什么是抢占式调度的基础

若用户程序不主动进入内核，定时器仍能周期性打断 CPU：

$User \rightarrow TimerInterrupt \rightarrow Kernel$

这使 OS 有机会：

- 记账；
- 更新运行时间；
- 判断时间片/优先级；
- 必要时触发抢占和重新调度。

10.4 设备中断：唤醒并不等于直接交 CPU

典型 I/O 完成：

$Device \rightarrow Interrupt \rightarrow KernelHandler \rightarrow Wakeup(Task)$

关键状态变化通常只是：

Blocked $\rightarrow$ *Ready*

之后是否立即抢占当前任务，取决于调度策略和系统状态。

11 综合题一：从 fork 到 wait 的进程生命周期

推荐课堂主线

不要先给学生五态图。先让学生跟踪一个真实进程：`shell fork` $\rightarrow$ `child exec` $\rightarrow$ `schedule` $\rightarrow$ `syscall` $\rightarrow$ `block` $\rightarrow$ `wakeup` $\rightarrow$ `exit` $\rightarrow$ `parent wait`。五态图应当从这个动态故事中“长出来”。

11.1 完整轨迹

1. Shell 正在 Running。
2. Shell 调用 `fork()` 进入内核。
3. 内核创建 child 的管理状态、地址空间语义（常用 COW）和继承资源引用。
4. child 进入 Ready；parent 也可能继续 Ready/Running，谁先运行不由 `fork()` 本身保证。
5. child 被调度后运行，调用 `exec()`。
6. `exec()` 替换 child 的程序映像，但保持该进程身份；未 `close-on-exec` 的 fd 可继续存在。
7. 新程序运行并发起一个系统调用。
8. 若系统调用可立即完成：`kernel` $\rightarrow$ `user`，仍是同一进程。
9. 若必须等待 I/O：`Running` $\rightarrow$ `Blocked`，进入等待队列，调度器选择另一个 Ready 实体。
10. 设备完成，中断进入内核；等待条件满足后 child：`Blocked` $\rightarrow$ `Ready`。
11. child 某次再次被调度：`Ready` $\rightarrow$ `Running`。
12. child 最终 `exit()`，释放大量运行资源，留下退出状态成为 zombie（Unix 语义）。
13. parent 的 `wait()` 获取退出信息并完成最终回收。

12 综合题二：一次阻塞 read 的“进程侧投影”

假设进程 A 调用：

```
read(fd, buf, 4096)
```

且底层条件使本次调用必须等待。这个专题只跟踪控制权与 task state：

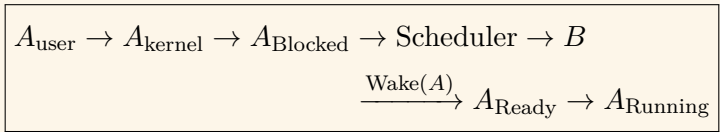
1. A 在用户态 Running。
2. `syscall` 使 CPU 进入 A 的内核上下文；此刻仍然是 A 在运行，不是已经切换进程。

- 3. 内核沿文件/I/O 子系统处理请求。若请求能立即完成，A 直接沿内核返回路径回到用户态，不发生 Blocked。
- 4. 本题设定必须等待：A 暂时不能继续，故 Running → Blocked，并挂到相应等待结构；
- 5. CPU 已经不能继续执行 A 的工作，scheduler 从 Ready 集合选择其他执行实体，必要时发生 context switch；
- 6. 后续底层事件完成后，相关子系统调用 wakeup，使 A：Blocked → Ready；
- 7. wakeup 不等于立刻获得 CPU。A 仍要等待低级调度；
- 8. A 再次被选中：Ready → Running，从原等待后的内核控制路径继续，最终完成 syscall 并返回用户态。

为什么这里故意不展开 fd/OFD/DMA/Page Cache

这些对象分别由文件系统与 I/O 专题拥有。OS-01/02 只需要知道：某个外部条件未满足会使当前 task block；条件完成会使它 wake 到 Ready；何时再次 Running 由调度决定。若题目要求完整追踪 read() 的文件对象、缓存、块映射、DMA 与完成中断，应切换到 OS 综合专题，把各 Topic 的局部模型串起来。

本专题真正要拿下的链



这条链同时检查 mode switch、block、wakeup、ready queue、调度机会与 context switch，但不把别的子系统细节重复背一遍。

13 线程实现模型：从“谁看得见”推出 ULT/KLT、M:N 与现代 Runtime

这一节只解决一个母问题

已经拥有多个用户态 Execution Context 后，它们怎样获得真正的 CPU 物理执行机会？
统一推演链条为：



以后所有 ULT/KLT、M:1/1:1/M:N、两级调度队列、阻塞传播、Scheduler Activations 以及 Go/Loom 现代运行时问题，全部从这条链推演，彻底告别零散优缺点的死记硬背。

13.1 可见性原则：谁调度，谁就必须“看见”被调度的对象

任何一级调度器（无论是用户态运行时调度器还是内核调度器）若要独立选择并推进一个对象，必须在自己的内存空间中掌握该对象的四个基本控制属性：

Schedulable Entity = Identity + Runnable/Blocked State + Saved Context + Scheduling Metadata

由此可直接推导出可见性铁律：

- 用户态线程库要调度逻辑线程 U ，就必须在用户态维护 User TCB 与用户执行现场；
- 内核调度器若要独立调度某条执行流，内核就必须为其分配对应的 Kernel Schedulable Context（内核 TCB、内核栈与调度属性）；
- **内核看不见纯用户级线程（ULT）**：因此内核不可能单独为某个 ULT 分配 CPU、不可能单独把某个 ULT 标记为 Ready/Blocked，也不可能跨 CPU 为 ULT 做系统级负载均衡。

13.2 ULT 与 KLT：不要问名字，问谁维护控制状态、谁执行选择

对比维度	用户级线程 (User-Level Thread, ULT)	内核级线程 (Kernel-Level Thread, KLT)
调度掌控者	用户态线程库 / 语言运行时 (Runtime)	OS 内核调度器
控制状态维护者	用户态运行时维护 User TCB	内核维护 Kernel TCB 与内核栈
内核透明度	内核完全不可见（内核仅感知其承载实体）	内核直接感知、命名并维护每条线程的运行状态
线程切换开销	极低：纯用户态切换寄存器与 SP，无模式切换	较低：需经中断/Trap 进入内核，由内核调度器完成切换
多核利用能力	取决于底层承载它的 KSE 数量；M:1 无法多核并行	极强：内核可将同进程的不同 runnable KLT 调度到不同 CPU 核心并行执行
阻塞调用的影响	取决于底层承载实体是否阻塞；M:1 下一个阻塞全停	细粒度隔离：一个 KLT 阻塞，同进程其他 KLT 仍可由内核调度运行
策略灵活性	极高：应用程序可按需定制专属调度算法	受限：必须遵循内核全局调度策略与优先级机制

“ULT 更公平/更不公平” 都不是定义性质

用户运行时可以在其内部线程集合中实现局部公平策略；但内核看不见单个 ULT，因而无法对其实施全系统公平保障。408 考题中若出现“ULT 公平性必然更优/更劣”，应警惕绝对化断言，必须回到**谁掌握全局信息、谁在分配什么资源**。

13.3 M:1、1:1、M:N 的本质：两层实体集合之间的映射

设某进程当前拥有 M 个用户逻辑执行上下文 $\{U_1, \dots, U_M\}$ ，内核为该进程提供了 N 个可独立调度的承载实体 $\{K_1, \dots, K_N\}$ ：

$$\{U_1, \dots, U_M\} \xrightarrow[\text{Runtime Policy}]{M:\{U\} \rightarrow \{K\}} \{K_1, \dots, K_N\} \xrightarrow[\text{Kernel Policy}]{\text{Dispatch}} \text{CPUs}$$

映射模型	映射结构	核心特征与运行表现	主要代价与核心缺陷
M:1	多个 ULT 复用单个内核实体 K_1	用户态切换极快（微秒/纳秒级）；内核状态极小	承载 K_1 一旦发生阻塞系统调用或硬缺页，全进程所有 ULT 失去执行载体；无法利用多核
1:1	每个 ULT 一一对应一个 KLT ($U_i \leftrightarrow K_i$)	细粒度并发控制；任意线程阻塞不影响其他线程；天然支持多核并行	创建、销毁与切换均需进入内核；高并发下内核 TCB 与内核栈内存开销大
M:N	多个 ULT 映射到较少或等量的一组 KLT ($M > N$)	兼具用户态轻量切换与多核并行能力；减少内核调度实体数量	两级调度器 (Two Scheduling Domains) 带来巨大的状态协同与通信复杂度

13.4 两级队列与 “Ready 必须带主语”

在 M:N 混合模型或现代用户运行时中，系统同时存在两级就绪队列：

$Q_U = \text{Runtime Runnable Queue (用户就绪队列)}$ 与 $Q_K = \text{Kernel Run Queue (内核可运行队列)}$

这导致了一个极为深刻、但传统五态模型无法表达的状态：

$U_2 \in Q_U \text{ (User-Ready),} \quad K_1 \notin Q_K \text{ (Kernel-Blocked)}$

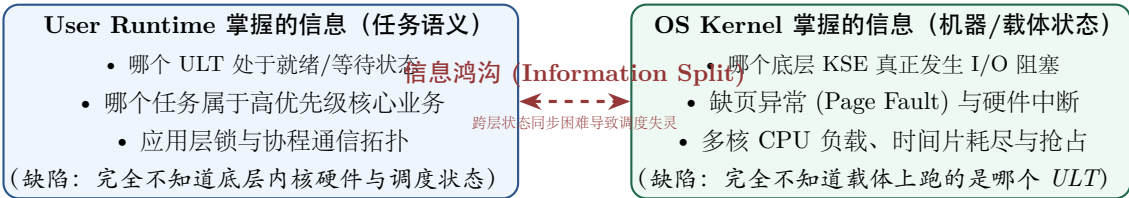
含义是：逻辑线程 U_2 在应用程序语义上已经准备好了一切数据与条件，但在底层暂时没有可用的内核承载实体（Carrier）将其送上 CPU。

状态判据铁律

以后做任何状态推演题，不要只问“它是不是 Ready? ”，而必须精确追问：
谁 Ready? 在哪个调度器的状态空间 (Q_U 还是 Q_K) 里 Ready?

13.5 为什么经典 M:N 在 OS 层面走向失败：信息分裂 (Information Split)

历史上 Unix/Solaris、FreeBSD 以及 Linux NGPT 曾尝试在操作系统层面实现通用的 M:N 线程模型，但最终工业界几乎全部退回到了 1:1（如 Linux NPTL）。其深层根本原因在于：信息被硬生生切成了两半（Information Split）：



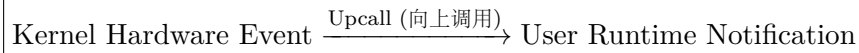
信息分裂引发三大经典顽疾：

1. **阻塞失察**：当 U_1 在 K_1 上执行阻塞系统调用时，内核只知道暂停 K_1 ，无法通知用户运行时将就绪的 U_2 换上；
2. **优先级反转与不一致**：内核可能将载有高优先级 ULT 的 K_1 抢占，却让载有低优先级 ULT 的 K_2 运行；

3. **信号处理混乱**：发给进程的信号究竟该由哪个 ULT 栈来承载执行，两层调度器难以达成低成本共识。

13.6 Scheduler Activations: 跨层通知的经典桥梁

为了克服信息分裂，1991 年经典论文提出了 **Scheduler Activations (调度器激活)** 机制。它建立了一座由内核向用户运行时主动通信的桥梁：

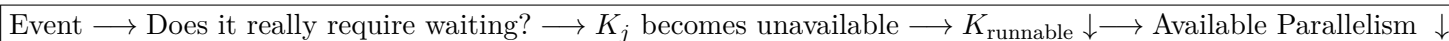


- 当某个承载实体 K_1 在内核中因 I/O 或缺页而阻塞时，内核并不静默等待，而是通过 **Upcall** 机制在另一个可运行的执行实体上唤起用户态运行时，通知它：“你的载体 K_1 阻塞了”；
- 用户运行时接获通知后，立刻从 Q_U 中挑选就绪的 U_2 绑定到新的载体上继续运行。

本质启示：一旦状态被切分在两个管理层次，就必须建立显式的跨层事件同步协议。这一思想在后来的虚拟机 Hypervisor、容器运行时以及现代 GPU 运行时中被反复复用。

13.7 阻塞传播与实际并行度上界公式

不要机械记忆“ULT 一阻塞全进程阻塞”。必须沿事件链做二阶判断：



1. 事件 $\neq$ 阻塞：

- Page Fault** 不天然等于 Blocked（次要缺页 Minor Fault 只需建立页表映射，无需磁盘 I/O，可立即在内核中恢复运行）；
- System Call** 不天然等于 Blocked（`getpid()` 或数据已在 Page Cache 的 `read()` 直接原路返回）。

2. **实际并行度上界公式**：在不考虑 CPU 亲和性绑核限制的 408 基础模型中，系统在某一时刻能够真正物理并行的执行流数量受三道闸门约束：

$$P_{\text{actual}} \leq \min \left(U_{\text{runnable}}, K_{\text{runnable}}, CPU_{\text{available}} \right)$$

此公式可一举秒杀所有多核与阻塞考题：

- M:1 模型中 $K_{\text{runnable}} \leq 1$ ，故即使 $CPU_{\text{available}} = 64$ 、 $U_{\text{runnable}} = 1000$ ， P_{actual} 最大也只能是 1；
- 1:1 模型中 $U_i \leftrightarrow K_i$ ，若有 2 个线程因 I/O 阻塞，则 K_{runnable} 减少 2，其余 runnable 线程继续占满剩余 CPU；
- M:N 模型中，若 3 个 KLT 中有 2 个阻塞，则 $K_{\text{runnable}} = 1$ ，瞬时最大并行能力降为 1。

13.8 现代 Runtime 的本质复活：执行上下文表示论 ($Object \neq Representation$)

现代编程语言（Go、Java 21 Loom、Rust、Erlang）并没有简单重复 1990 年代 OS 级 M:N 的老路，而是实现了更高层次的进化。其核心心智模型是：

Execution Context (执行上下文是语义) $\neq$ Native Stack (不等于某一种具体的连续机器栈)

只要能满足“在未来任意时刻准确恢复执行现场 (Enough State to Resume)”，执行上下文在内存中可以有完全不同的物理表示：

并发抽象技术	物理内存表示 (Representation)	切换机制与承载方式
Native Thread (OS 1:1)	固定的连续 OS 内核/用户栈 (如 2MB/8MB) + 寄存器	内核上下文切换，切换内核栈与通用寄存器现场
Go Goroutine	极小连续堆栈 (初始仅 2KB, 可动态扩容)	Go Runtime 在用户态通过 <code>runtime.gogo</code> 切换 SP/PC, 借助 GMP 模型与 Work-Stealing 调度
Java 21 Virtual Thread	堆上的 Continuation 冻结帧 (Yieldable)	挂载到 Carrier 线程 (ForkJoinPool) 执行; 阻塞时 Unmount 卸下, 就绪时 Mount 重新挂载
Rust Async C++20	/ 编译器自动生成的有限状态机 (State Machine)	无栈协程 (Stackless); 状态机保存在 Future/Task 堆对象中, 由 Executor 轮询 (Poll)

现代 M:N 真正做对了什么

现代 Runtime 解决阻塞的核心不是硬去调停两级内核调度器，而是通过**异步 I/O 多路复用 (epoll/io_uring)** 将传统的线程阻塞操作转化为用户态协程的挂起让出：

Blocking Operation $\xrightarrow{\text{Runtime Interception}}$ Park(U_i) (Carrier K 不阻塞，立刻切换执行 U_{i+1})

从而既享受了百万级逻辑执行流 (U) 的超低内存与用户态切换优势，又彻底避开了底层操作系统 Carrier (K) 的被动阻塞。

14 408 解题检查表：从概念辨析到状态推演

14.1 线程做题九问控制器 (Thread Control Adapter)

遇到任何线程共享、ULT/KLT、M:N、阻塞传播、多核并行或线程切换考题，按以下九步严格推演：

1. 当前对象处于哪一层？区分 Process Resource Context (R)、User Execution Context (U)、Kernel Schedulable Context (K)、Thread Group (G) 还是物理 CPU (C)？
2. 这条执行流属于哪个资源世界？其虚拟地址空间（页表基址）、代码段、数据段、堆与打开文件表属于哪个进程？
3. 哪些状态是 **shared**，哪些状态是 **per-thread**？牢记：代码/堆/全局变量/文件表共享；PC/寄存器/SP/栈帧/TLS/信号屏蔽字独立私有。
4. 题目问“私有/共享”时是在问 **Ownership**、**Addressability** 还是 **Concurrency Safety**？所有权归线程上下文；同一地址空间内其他线程只要拿到指针就能直接访问（无硬件隔离）；但并发修改必须加锁同步。
5. 谁能看见并调度它？用户态运行时维护 User TCB 调度 U ；内核调度器维护 Kernel TCB 调度 K 。内核看不见纯 ULT。

6. **U 到 K 的映射模型是什么？** M:1、1:1 还是 M:N? 两级队列 Q_U 与 Q_K 的容量关系如何?
7. **当前状态属于哪一级？发生的事件真正导致等待了吗？** User-Ready 还是 Kernel-Blocked? Syscall 与 Page Fault 不天然等于 Blocked; 若阻塞, 标出哪个底层 Carrier 失去运行资格。
8. **事件的作用域 (Event Scope) 是单线程还是整个线程组？** 单线程退出 (`pthread_exit`) vs 进程级退出 (`exit`); 线程定向信号 (SIGSEGV) vs 进程定向信号 (SIGINT)。
9. **若发生切换, 究竟换了什么？还剩多少物理并行算力？** Mode Switch、Execution Context Switch、Memory Context Switch 还是 Mapping Change? 用 $P_{\text{actual}} \leq \min(U_{\text{runn}}, K_{\text{runn}}, CPU_{\text{avail}})$ 精确计算剩余并行度。

线程题的最短定性口诀

先分层, 再看谁能看见; 先找承载, 再谈阻塞; 先定调度实体, 再谈时间片。

14.2 进程状态题八问

1. 当前实体属于 New、Ready、Running、Blocked、Exit 中哪一类基本状态?
2. 题目是否额外引入 Suspend? 若引入, 它现在是 Ready-Suspend 还是 Blocked-Suspend?
3. 它当前缺的是 CPU、外部事件, 还是“重新激活/换入”条件?
4. 发生了什么事件? 这条状态箭头是否有明确事件依据?
5. 若等待事件发生, Blocked-Suspend 应先到 Ready-Suspend 还是可以直接 Running?
6. 这次变化涉及高级、中级还是低级调度?
7. 事件只是改变 task state, 还是同时产生调度机会?
8. 是否混淆“进程在内核态执行”和“已经换成另一个进程”?

14.3 调度计算题九步

1. 先判断题目讨论高级/中级/低级调度中的哪一层; 经典算法计算题通常是低级 CPU 调度;
2. 圈出题目真正优化的目标: throughput/turnaround、response/fairness, 还是 deadline/predictability;
3. 写出 arrival time、burst/service time、priority、quantum 等输入;
4. 确定算法是抢占还是非抢占;
5. 每个真正的决策事件点重新确定当前 ready set;
6. 若是 HRRN, 只在需要重新选任务时计算 $R = 1 + W/S$;
7. 按策略选择, 并明确同值时题目采用什么 tie-break;
8. 画 Gantt 图并逐段检查到达、完成、抢占事件;
9. 从完成时刻反推 turnaround / waiting / response, 最后检查单位与定义。

14.4 系统调用/中断题九问

1. 入口是 syscall、exception 还是 interrupt?
2. 是同步事件还是异步事件?
3. 若题目问中断响应, 哪些是硬件自动入口动作, 哪些由软件 ISR 保存/恢复?
4. CPU 是否发生 privilege/mode transition?
5. 当前是否仍处于原进程的执行上下文?
6. 当前任务处理完事件后还能继续吗?
7. 若不能, 它进入哪个等待队列/状态?
8. 即使产生了调度请求, 当前是否已经到允许 context switch 的安全点?
9. 最终是 return same task, 还是 scheduler 选择 another task?

14.5 PCB / TCB / 进程资源接口题八问

1. 题目问的是进程身份、CPU 执行现场、调度状态, 还是资源引用? 先把 PCB 信息类别定位清楚;
2. 当前信息属于整个进程的 resource/protection context, 还是某一条线程的 execution context? 若是后者, 优先按 TCB 语义定位;
3. 需要恢复执行的是 PC/寄存器/栈等 execution context, 还是进程级 resource context?
4. PCB/任务结构保存的是对象本体, 还是能找到对象的引用与管理信息?
5. fork() 后哪些进程管理状态新建, 哪些资源引用按语义继承?
6. 若代码根据 fork() 返回值分支, 父进程得到 child PID、子进程得到 0 后, 各自会走哪条控制流?
7. exec() 是创建新进程, 还是在同一进程身份下替换程序映像?
8. 如果题目继续追问 offset、OFD、inode、unlink 等文件内部语义, 立即切换到 OS-06/07, 不在本专题继续猜。

14.6 IPC 题六问

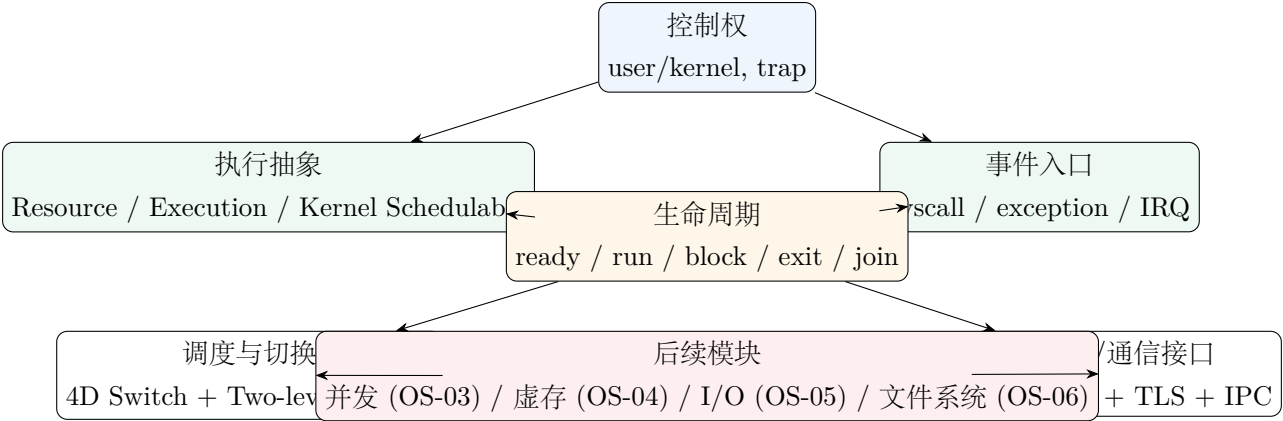
1. 两个进程之间要传递的是共享状态、数据流/消息, 还是事件通知?
2. 承载通信的对象在哪里: 共享映射、pipe buffer、message queue/mailbox, 还是 notification state?
3. 当前哪一方缺少什么条件, 因此可能 Blocked?
4. 哪个事件改变了通信对象状态, 并使 waiter 从 Blocked 变成 Ready?
5. 如果是共享内存, 题目是否已经从“能通信”进入了“怎样同步”? 若是, 切到 OS-03;
6. 如果继续追问映射、OFD/inode 或 transport socket 内部语义, 是否应该切到 OS-04、OS-06/07 或 Network?

15 原笔记与教材中值得保留、但必须升格的内容

原教材/直觉表述	局限与盲区评价	Canonical 升格心智模型
程序 → 进程 → 线程	过于线性，混淆了静态程序、动态实例、资源环境与执行流	形式化为 $\text{Process Instance} = (G, R, \{U_i\})$ ，并锁死 $\text{Program} \neq \text{Process Image} \neq \text{Process Instance} \neq \text{Thread}$ 的对象/表示边界
进程 = 资源分配单位，线程 = 调度单位	在纯 ULT 与协程下破产，语焉不详	拆分: $\text{Execution Context} \neq \text{Kernel Schedulable Context}$ ，先定调度域
“线程栈私有，其他线程绝对不可访问”	混淆了语义归属与硬件内存保护	确立三权分立: $\text{Ownership} \neq \text{Addressability} \neq \text{Concurrency Safety}$
“线程切换比进程快，因为不刷 TLB”	过度单一化，忽略了体系结构深层开销	建立四维模型与 $C_{\text{switch}} = C_{\text{direct}} + C_{\text{memory}} + C_{\text{locality}}$ (Cache 冷失效)
“时间片属于线程”	仅在 1:1 KLT 下成立，缺乏一般性	CPU 时间预算严格属于该调度器所能看见的可调度实体
“ULT 一个阻塞，全进程阻塞”	忽略了 M:1 前提与“是否真正需要等待”	建立 $\text{Event} \rightarrow \text{Need Wait?} \rightarrow K_{\text{runnable}} \downarrow \rightarrow P_{\text{actual}}$ 判定链
“ULT 公平性更好/更差”	混淆了应用局部调度与系统全局调度	公平性必须显式写出 $\text{Subject} + \text{Entitlement}$; 区分 $\text{Process/User/Thread fairness}$ 与两层调度域
“调度器选中”=“新任务已经运行”	混淆 Decision 与 Handoff	拆成 $\text{Enqueue} \rightarrow \text{Scheduler Pick} \rightarrow \text{Dispatcher/Switch} \rightarrow \text{Running}$ ，显式保留 dispatch latency
“多级队列”和“多级反馈队列”只是名字不同	没看见队列成员关系是否可变	MLQ: membership fixed; MLFQ: membership 可由 feedback 改变，并把问题拆成分类/队列间/队列内三次决策
“多核调度只是把 CPU 数量改大”	忽略了任务放在哪个 CPU、迁移与局部性	把 C 升格为 $\text{Who} + \text{Where}$; 显式权衡 $\text{global/per-CPU queues, load balance, affinity}$ 与 migration cost
“进程上下文就是寄存器”	只看到了必须搬动的现场	教材三层 Context 负责列状态，四维 Switch 负责判定本次究竟改变 $\text{Privilege/Execution/Memory/Mapping}$ 哪几维
五态模型直接套用所有系统	无法表达 M:N 下两级状态异步	引入两级队列 Q_U, Q_K ，“就绪必须带主语/调度域”
“Scheduler Activations 是失败的历史八卦”	未看清跨层状态同步的架构原型	升格为 $\text{Kernel Event} \xrightarrow{\text{Upcall}} \text{Runtime}$ 跨层状态通信范式
“Go/Loom/Rust 只是语言级库语法”	未看清执行上下文的统一本质	确立执行上下文表示论: $\text{Execution Context} \neq \text{Native Stack}$ (对象 $\neq$ 表示)
“join(T) 是进程/线程间通信”	混淆了数据通信与生命周期同步	明确定义为生命周期状态同步原语: $\text{WaitUntil}(\text{State}(T) = \text{Terminated})$

“线程共享文件描述符”	仅停留在整数层面，未穿透到 VFS	穿透到 VFS：共享 fd 意味着共享底层 Open File Description（f_pos 等下游可变状态）
-------------	-------------------	--

16 知识树与专题接口



本专题终极心智模型

这一模块学完，看到“进程/线程”时不应再停留在死记硬背，而应在脑海中自动浮现包含 9 个要素的清晰关系图：

$$S_{\text{exec}} = (G, R, U, K, M, Q_U, Q_K, W, C)$$

1. 看到“进程”时自动问：

这是哪个实例 G ？绑定哪个资源世界 R ？内部有哪些执行流 $\{U_i\}$ ？

2. 看到“线程”时自动问：

共享哪个资源世界 R ？独立现场 U_i 是什么？谁能看见并调度它？

3. 看到“状态变化”时自动问：

什么事件？哪个层次的对象（ U 还是 K ）变状态？当前还能继续吗？

4. 看到“调度与多核”时自动区分：

Mechanism $\neq$ Policy, Execution Context $\neq$ Kernel Schedulable Context

并用 $P_{\text{actual}} \leq \min(U_{\text{runnable}}, K_{\text{runnable}}, CPU_{\text{available}})$ 锁定实际物理算力；

5. 看到“切换”时自动拆为四维：

Privilege / Execution / Memory / Mapping

6. 看到 read/fork/exec/pthread_create/join 时，严格沿控制权、task state、IPC 进程侧状态与资源引用交接完整推演。

参考与工程边界来源

- Remzi H. Arpaci-Dusseau, Andrea C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*: 进程、受限直接执行、调度与并发的教学框架。
- MIT PDOS, *xv6: a simple, Unix-like teaching operating system*: trap、scheduler、sleep/wakeup、process lifecycle 的实现级案例。
- POSIX / The Open Group: `fork()`、`exec()`、`pthread_create()`、`pthread_join()` 规范语义。
- POSIX / Linux man-pages *pthreads(7)*, *signal(7)*, *clone(2)*: 进程内线程共享属性与独立属性、信号路由规则与线程组模型。
- Anderson, Bershad, Lazowska, Levy, *Scheduler Activations: Effective Kernel Support for the User-Level Management of Parallelism* (ACM TOCS 1992): 两级调度与跨层状态通知的经典理论。
- OpenJDK JEP 444, *Virtual Threads* (Java 21): 虚拟线程、Continuation 与 Carrier 线程映射的现代运行时案例。
- Linux Kernel Documentation, *EEVDF Scheduler*: 以内核 runnable task 为调度对象维护 virtual runtime、lag 与 virtual deadline 的工程实现边界。